

**PENGEMBANGAN GRAPHRAG FRAMEWORK
BERBASIS PENGETAHUAN KOMUNITAS STACK
OVERFLOW UNTUK MEREDUKSI HALUSINASI
PADA LLM**

TESIS

Oleh

Dartika Anie Marian
23524319



**PROGRAM STUDI MAGISTER INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Mei 2026**

LEMBAR PENGESAHAN

Pengembangan GraphRAG Framework Berbasis Pengetahuan Komunitas Stack Overflow untuk Mereduksi Halusinasi pada LLM

TESIS

Oleh

Dartika Anie Marian
23524319

Program Studi Magister Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Tesis ini telah disetujui dan disahkan
di Bandung, pada tanggal 29 Mei 2026

Pembimbing

Dr.Ir. Arry Akhmad Arman, MT.

NIP. 196504141991021001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tesis ini adalah hasil karya saya sendiri yang dibuat dengan sebenar-benarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan tesis ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan tesis.
3. Isi tesis ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa dokumen tesis ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 29 Mei 2026

Dartika Anie Marian

NIM: 23534219

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan tesis ini, saya telah menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu. Berikut adalah rincian penggunaan AI dalam penulisan tesis ini.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Microsoft Copilot	Sedang	Bab II dan Subbab III.2 hal. 34
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	...	...	...
5	Penyusunan bahan diskusi	...	...	...
6	Penyusunan kode program	...	...	...
7	Penyusunan formula atau perhitungan matematis	...	...	...
8	Penyusunan konsep desain atau algoritma	...	...	...
9	Penyusunan analisis data	...	...	...
10	Penyusunan kesimpulan atau rekomendasi	...	...	...

Bandung, 29 Mei 2026

Dartika Anie Marian

NIM: 23524319

ABSTRAK

Pengembangan GraphRAG Framework Berbasis Pengetahuan Komunitas Stack Overflow untuk Mereduksi Halusinasi pada LLM

oleh

Dartika Anie Marian

23524319

Kata kunci:

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan Tesis yang berjudul "Pemodelan Machine Learning untuk Prioritisasi dan Estimasi Kebutuhan Fasilitas Kesehatan Kota Bandung". Tesis ini disusun sebagai salah satu syarat untuk menyelesaikan program Magister Informatika di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Bandung, 20 Mei 2026

Penulis

Dartika Anie Marian

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	<i>Artificial Intelligence</i>	1
CNN	<i>Convolutional Neural Network</i>	2
GPU	<i>Graphics Processing Unit</i>	14

BAB I

PENDAHULUAN

I.1 Latar Belakang

Perkembangan *Large Language Model* (LLM) dalam beberapa tahun terakhir telah mendorong transformasi signifikan pada berbagai aplikasi kecerdasan buatan, termasuk natural language understanding, text generation, conversational agents, machine translation, serta code generation (**kumar2024**; **qin2025**). Model-model seperti GPT-4 dan LLaMA 2 telah menunjukkan kemampuan luar biasa dalam memahami dan menghasilkan teks natural yang menyerupai tulisan manusia. **Haque2025**<empty citation> menegaskan bahwa LLM bukan hanya mengubah cara perangkat lunak dikembangkan, tetapi secara fundamental mendefinisikan ulang peran pengembang dalam siklus hidup rekayasa perangkat lunak (*software engineering*). Adopsi LLM di industri perangkat lunak juga meningkat secara signifikan, sebagaimana dikonfirmasi oleh **Ronanki2026**<empty citation> yang melakukan studi multi-kasus pada tiga organisasi yang menggunakan LLM dalam aktivitas rekayasa perangkat lunak dan mengidentifikasi perlunya praktik terbaik yang dapat ditindaklanjuti untuk adopsi LLM yang efisien dan bertanggung jawab. Meskipun demikian, di balik kemampuan generatif tersebut, terdapat permasalahan mendasar yang belum sepenuhnya terpecahkan yaitu kecenderungan LLM untuk menghasilkan informasi yang tampak meyakinkan namun faktanya tidak akurat atau bahkan sepenuhnya merupakan fabrikasi (**Ji2023**).

Fenomena ini dikenal sebagai *hallucination* (halusinasi) dan menjadi salah satu hambatan paling kritis dalam penerapan LLM pada domain yang membutuhkan keakuratan tinggi. **Ji2023**<empty citation> mendefinisikan halusinasi sebagai kondisi di mana model menghasilkan keluaran yang tidak sesuai dengan fakta yang dapat diverifikasi, tidak konsisten dengan konteks yang diberikan, atau tidak memiliki dasar empiris yang dapat ditelusuri. **Huang2025**<empty citation> dalam survei komprehensifnya di *ACM Transactions on Information Systems* mengidentifikasi tiga lapisan akar penyebab halusinasi yang saling terkait: (1) kualitas dan kelengkapan data pelatihan yang tidak merata, (2) keterbatasan proses penyelarasan (*alignment*) model terhadap pengetahuan faktual, dan (3) mekanisme inferensi yang cenderung menghasilkan teks yang terdengar koheren tanpa jaminan akurasi faktual. **Chang2026**<empty citation> melengkapi analisis ini dengan mengidentifikasi tiga

tantangan fundamental dalam generasi teks berbasis LLM yang memperkuat kecenderungan halusinasi: (1) pemahaman instruksi yang tidak memadai pada konteks panjang dan percakapan multi-gilir, (2) penalaran intensi yang tidak konsisten, khususnya ketika instruksi mengandung informasi ambigu, dan (3) generasi dialog yang tidak andal (*unreliable dialogue generation*) yang menghasilkan konten tidak stabil dari prompt yang serupa.

Dalam konteks pemrograman dan rekayasa perangkat lunak, halusinasi LLM bermanifestasi dalam berbagai bentuk yang berpotensi menimbulkan dampak langsung terhadap kualitas sistem. **Alomari2026**<empty citation> dalam tinjauan literatur sistematis terhadap 49 studi penggunaan LLM untuk peningkatan kualitas kode menemukan bahwa kode yang dihasilkan atau direfaktorisasi oleh LLM tidak dapat diandalkan dan masih menghadapi tantangan besar dalam hal optimasi dan keandalan. Sebuah temuan yang mengonfirmasi bahwa pengetahuan parametrik LLM saja tidak cukup untuk menjamin keakuratan di domain teknis yang terus berkembang. **DaSilva2025**<empty citation> memperkuat bukti ini melalui studi empiris yang membandingkan jawaban ChatGPT dan LLaMA terhadap pertanyaan Stack Overflow. Hasil penelitian **DaSilva2025**<empty citation> menunjukkan bahwa meskipun LLM menantang keahlian manusia, LLM tidak melampaui keahlian komunitas manusia untuk sejumlah domain tertentu. Yang lebih signifikan, **DaSilva2025**<empty citation> mendokumentasikan penurunan nyata aktivitas *posting* pengguna di Stack Overflow sejak hadirnya ChatGPT. Pengguna mulai mengandalkan LLM meskipun keandalannya belum terjamin penuh, sehingga berpotensi memperparah penyebaran informasi teknis yang tidak akurat.

Stack Overflow yang merupakan platform utama pengembang untuk pertanyaan seputar pemrograman dan pengembangan perangkat lunak, kini berada di persimpangan antara ancaman dan peluang akibat kehadiran LLM. Di satu sisi, aktivitas penggunaannya menurun (**DaSilva2025**). Di sisi lain, komunitas Stack Overflow menyimpan lebih dari 58 juta pasang pertanyaan dan jawaban yang telah melewati mekanisme kurasi ketat berupa sistem *upvote/downvote*, penanda jawaban yang diterima (*accepted answer*), dan sistem reputasi kontributor (**Kabir2024**). Sinyal kepercayaan komunitas ini menjadikan Stack Overflow bukan sekadar repositori teks, melainkan basis pengetahuan tervalidasi secara sosial yang potensial digunakan untuk memberikan landasan pengetahuan bagi jawaban LLM berdasarkan fakta yang telah diverifikasi oleh praktisi domain. **Nashid2023**<empty citation> menunjukkan bahwa pemanfaatan konten Stack Overflow sebagai sumber pengambilan konteks secara signifikan meningkatkan kualitas jawaban LLM pada tugas-tugas terkait ko-

de, menggarisbawahi potensi besar repositori ini.

Salah satu pendekatan yang paling menjanjikan untuk memitigasi halusinasi adalah *Retrieval-Augmented Generation* (RAG) (Lewis2020). YizhengHuang2026<empty citation> dalam survei mereka di *ACM Computing Surveys* menjelaskan bahwa RAG menggabungkan teknik pengambilan informasi (*information retrieval*) dengan kemampuan generatif LLM untuk mengatasi keterbatasan pengetahuan statis model, sehingga memungkinkan integrasi informasi eksternal yang dinamis dan terkini pada saat inferensi. DelaCruzCabello2025<empty citation> melalui tinjauan literatur sistematis pada domain deteksi anomali log sistem komputer menemukan bahwa RAG secara substansial meningkatkan akurasi dan interpretabilitas jawaban LLM tanpa memerlukan *fine-tuning* yang ekstensif, dan bahwa integrasi RAG dengan LLM menunjukkan kemampuan adaptasi yang kuat terhadap lingkungan operasional yang berubah. Temuan ini mengkonfirmasi potensi RAG sebagai strategi mitigasi halusinasi yang praktis dan skalabel di berbagai domain teknis.

Meskipun demikian, implementasi RAG konvensional yang menggunakan pencarian berbasis kemiripan vektor (*dense retrieval*) terhadap potongan-potongan teks yang tidak terstruktur memiliki keterbatasan mendasar yaitu pendekatan ini gagal memanfaatkan relasi semantik kompleks dan hubungan kontekstual antar konsep yang justru sangat penting dalam domain pemrograman (YizhengHuang2026). Sebagai respons atas keterbatasan ini, pendekatan *Graph-based Retrieval-Augmented Generation* (GraphRAG) diusulkan untuk mengintegrasikan struktur *knowledge graph* ke dalam proses pengambilan konteks. Zhu2026<empty citation> dalam survei komprehensif mereka di *ACM Computing Surveys* mengidentifikasi tiga inovasi kunci GraphRAG dibandingkan RAG konvensional: (1) representasi pengetahuan berbasis graf yang secara eksplisit mengkodekan relasi antar entitas dan hierarki domain, (2) teknik pengambilan berbasis graf yang memungkinkan pengambilan konteks dengan kemampuan penalaran multi-hop, dan (3) algoritma integrasi pengetahuan yang sadar-struktur untuk menghasilkan teks yang lebih akurat dan koheren. Pan2024<empty citation> dalam menegaskan bahwa penggabungan LLM dengan *knowledge graph* membentuk paradigma baru yang secara signifikan meningkatkan kemampuan penalaran berbasis pengetahuan dan mengurangi fabrikasi faktual (*factual hallucination*).

Dari sisi evaluasi, Gu2026<empty citation> dalam survei komprehensif mereka tentang *LLM-as-a-Judge* menegaskan bahwa mengukur keandalan LLM merupakan tantangan signifikan yang membutuhkan desain dan standarisasi yang cermat.

Mereka mengusulkan strategi untuk meningkatkan konsistensi evaluasi dan memitigasi bias, sebuah kerangka yang relevan untuk mengukur efektivitas reduksi halusinasi pada sistem berbasis GraphRAG dan menjadi prasyarat agar kontribusi sebuah *framework* mitigasi halusinasi dapat diklaim secara akademis.

Meskipun kemajuan di masing-masing bidang LLM, RAG, GraphRAG, dan penambahan pengetahuan komunitas cukup pesat, terdapat kesenjangan metodologis yang signifikan ketika keempat komponen ini dipertimbangkan secara terpadu. Pertama, penelitian yang secara langsung membandingkan LLM dengan Stack Overflow seperti **DaSilva2025**<empty citation> dan **Kabir2024**<empty citation>, bersifat evaluatif. Mereka mengukur dan mendokumentasikan kesenjangan kualitas antara LLM dan keahlian komunitas, tetapi tidak mengusulkan mekanisme arsitektural untuk menjembatani kesenjangan tersebut. *Kedua*, penelitian yang menggunakan konten Stack Overflow sebagai sumber konteks untuk LLM, seperti **Nashid2023**<empty citation> hanya menerapkan RAG konvensional berbasis kemiripan vektor dan tidak memanfaatkan struktur relasional pengetahuan komunitas maupun sinyal kepercayaan komunitas (*upvote*, *accepted answer*) sebagai bobot kepercayaan dalam proses pengambilan konteks. *Ketiga*, meskipun pendekatan GraphRAG terbukti superior dibandingkan RAG konvensional pada berbagai domain (**Zhu2026**), serta integrasi *knowledge graph* dengan LLM diyakini sebagai paradigma yang menjanjikan (**Pan2024**), belum ada penelitian yang secara khusus membangun *knowledge graph* dari konten Stack Overflow yang mengkodekan sinyal kepercayaan komunitas untuk digunakan sebagai fondasi GraphRAG pada domain pemrograman. Sementara itu, tantangan halusinasi akibat keterbatasan pemahaman instruksi dan generasi dialog yang tidak andal (**Chang2026**; **Huang2025**) tetap menjadi masalah terbuka yang membutuhkan pendekatan grounding berbasis pengetahuan terverifikasi.

Kesenjangan-kesenjangan ini secara kolektif membuka peluang kontribusi yang signifikan: pengembangan sebuah *GraphRAG Framework* yang secara sistematis (1) membangun *knowledge graph* dari konten Stack Overflow yang mengkodekan entitas teknis, relasi semantik, dan sinyal kepercayaan komunitas sebagai bobot kepercayaan eksplisit; (2) merancang mekanisme *graph-guided retrieval* yang memanfaatkan struktur relasional tersebut untuk mengidentifikasi konteks yang lebih relevan dibandingkan RAG konvensional; (3) mengintegrasikan konteks hasil traversal ke dalam proses inferensi LLM; dan (4) mengukur efektivitas reduksi halusinasi secara kuantitatif menggunakan protokol evaluasi yang terstandarisasi.

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, kondisi ideal yang diharapkan adalah tersedianya mekanisme *grounding* berbasis pengetahuan komunitas yang tervalidasi secara sosial sehingga LLM dapat menghasilkan jawaban teknis yang akurat dan dapat dipertanggungjawabkan. Namun, kondisi aktual yang dijumpai menunjukkan kesenjangan metodologis yang signifikan: penelitian yang menggunakan konten Stack Overflow sebagai konteks LLM masih menggunakan RAG konvensional dan tidak memanfaatkan struktur relasional pengetahuan komunitas (Nashid2023); pendekatan GraphRAG yang ada belum mengeksplorasi sumber pengetahuan bervalidasi sosial maupun domain pemrograman secara spesifik (Zhu2026; Pan2024); sementara itu, tantangan halusinasi dalam generasi teks LLM pada instruksi yang ambigu dan konteks panjang masih belum terpecahkan (Chang2026; Huang2025). Kesenjangan antara kondisi ideal dan kondisi aktual tersebut menjadi landasan permasalahan utama dalam penelitian ini, yang dirumuskan sebagai berikut.

Bagaimana merancang dan mengembangkan GraphRAG *Framework* berbasis pengetahuan komunitas Stack Overflow yang secara terukur mampu mereduksi halusinasi pada *Large Language Model* di domain pemrograman?

Untuk menjawab pertanyaan utama tersebut, penelitian ini mengajukan pertanyaan turunan sebagai berikut.

1. Bagaimana membangun *knowledge graph* yang merepresentasikan pengetahuan teknis komunitas Stack Overflow secara terstruktur, termasuk entitas (konsep, fungsi, pustaka, *error*), relasi antar entitas, serta sinyal kepercayaan komunitas (*upvote*, *accepted answer*, skor reputasi) sebagai bobot kepercayaan eksplisit pada simpul dan sisi graf?
2. Bagaimana merancang mekanisme *graph-guided retrieval* yang mampu mengidentifikasi subgraf relevan berdasarkan kueri pengguna, sedemikian sehingga konteks yang diekstraksi lebih kaya secara semantik dan relasional dibandingkan pengambilan berbasis kemiripan vektor pada RAG konvensional?
3. Bagaimana mengintegrasikan konteks hasil traversal *knowledge graph* Stack Overflow ke dalam proses inferensi LLM sehingga jawaban yang dihasilkan memiliki tingkat akurasi faktual yang lebih tinggi dibandingkan LLM tanpa mekanisme *grounding*?
4. Bagaimana merancang protokol evaluasi yang mampu mengukur dan memvalidasi efektivitas *framework* yang diusulkan dalam mereduksi halusinasi LLM menggunakan metrik kuantitatif yang terstandarisasi, serta membandingkan

kinerja *framework* terhadap dua kondisi *baseline*: LLM murni dan RAG konvensional?

I.3 Tujuan

Tujuan penelitian ini dibagi menjadi tujuan umum dan tujuan khusus. Tujuan umum penelitian ini adalah mengembangkan *GraphRAG Framework* berbasis pengetahuan komunitas Stack Overflow sebagai artefak penelitian desain yang mampu mereduksi halusinasi LLM pada domain pemrograman secara terukur, sekaligus menghasilkan kontribusi pengetahuan berupa prinsip-prinsip desain yang dapat digunakan kembali oleh penelitian serupa di domain teknis lainnya.

Adapun tujuan khusus dari penelitian ini adalah sebagai berikut.

1. Membangun *knowledge graph* dari data Stack Overflow yang merepresentasikan entitas teknis, relasi semantik antar entitas, dan sinyal kepercayaan komunitas sebagai bobot pada simpul dan sisi graf.
2. Merancang dan mengimplementasikan modul *graph-guided retrieval* yang memanfaatkan traversal *knowledge graph* untuk mengekstraksi konteks yang relevan secara semantik dan relasional berdasarkan kueri pengguna, melampaui keterbatasan RAG konvensional berbasis kemiripan vektor.
3. Mengembangkan modul *grounded generation* yang mengintegrasikan konteks hasil traversal graf ke dalam mekanisme *prompting* LLM untuk menghasilkan jawaban teknis yang memiliki *grounding* faktual lebih kuat dibandingkan LLM tanpa mekanisme pengambilan konteks eksternal.
4. Melakukan evaluasi untuk mengukur tingkat reduksi halusinasi *framework* yang diusulkan menggunakan metrik kuantitatif, serta membandingkan hasilnya terhadap dua *baseline* yaitu LLM murni dan RAG konvensional.

I.4 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat pada dua dimensi yang saling melengkapi, yaitu manfaat teoritis dan manfaat praktis.

Manfaat Teoritis

1. **Model arsitektur GraphRAG untuk domain *question answering*.** *Framework* yang diusulkan memberikan kontribusi berupa model arsitektur GraphRAG yang secara eksplisit mempertimbangkan validasi sosial komunitas sebagai dimensi kepercayaan dalam proses pengambilan konteks. Model ini memperkaya literatur akademik pada persimpangan antara *information retrieval*, *knowledge graph*, dan *large language model*.
2. **Kerangka evaluasi halusinasi yang terstandarisasi.** Protokol evaluasi yang

dirancang dalam penelitian ini mencakup metrik akurasi faktual, konsistensi, dan tingkat halusinasi dalam tiga kondisi perbandingan, dapat menjadi referensi metodologis bagi penelitian mitigasi halusinasi LLM di berbagai domain aplikasi.

Manfaat Praktis

1. **Peningkatan keandalan bantuan pemrograman berbasis LLM.** *GraphRAG Framework* yang dihasilkan dapat diintegrasikan ke dalam alat bantu pengembangan (*developer tools*) berbasis LLM, seperti *AI coding assistant*, untuk menghasilkan saran kode dan penjelasan teknis yang lebih akurat dan dapat ditelusuri sumbernya.
2. **Panduan adopsi LLM yang bertanggung jawab di industri perangkat lunak.** Temuan penelitian ini memberikan bukti empiris dan panduan arsitektural bagi organisasi perangkat lunak yang ingin mengadopsi LLM secara efisien dan bertanggung jawab, khususnya dalam mengelola risiko penyebaran informasi teknis yang tidak akurat.
3. **Pemanfaatan kembali aset pengetahuan komunitas Stack Overflow.** Penelitian ini menunjukkan cara memanfaatkan lebih dari 58 juta pasang pertanyaan dan jawaban Stack Overflow yang telah tervalidasi secara sosial sebagai fondasi pengetahuan yang terstruktur, sehingga aset komunitas ini tidak hanya menjadi repositori pasif tetapi juga menjadi infrastruktur *grounding* aktif untuk sistem AI generatif.

I.5 Ruang Lingkup dan Batasan Masalah

Penelitian ini berfokus pada perancangan dan pengembangan *GraphRAG Framework* untuk domain pemrograman dengan menggunakan data dari komunitas Stack Overflow sebagai sumber pengetahuan utama. Evaluasi dilakukan pada pertanyaan dan jawaban teknis yang mencakup topik-topik pemrograman umum (*general programming*) seperti Python, JavaScript, dan Java, yang merupakan *tag* dengan volume konten tertinggi di Stack Overflow.

Batasan masalah:

1. *Framework* yang dikembangkan difokuskan pada domain pemrograman dan tidak dimaksudkan untuk langsung digeneralisasi ke domain non-teknis tanpa adaptasi pada skema *knowledge graph* dan protokol evaluasinya.
2. Pembangunan *knowledge graph* dibatasi pada ekstraksi entitas dan relasi dari teks pertanyaan, jawaban, dan komentar Stack Overflow, tanpa menyertakan eksekusi atau validasi kode secara dinamis.

3. Penelitian ini tidak membahas aspek *fine-tuning* LLM. Seluruh eksperimen menggunakan LLM dalam mode *inference* tanpa modifikasi parameter model.
4. Pengukuran halusinasi dibatasi pada dimensi akurasi faktual dan konsistensi informasi, tidak mencakup dimensi bias, toksisitas, atau kesesuaian gaya bahasa.
5. Penelitian ini tidak mencakup aspek skalabilitas dan *deployment framework* ke lingkungan produksi. Seluruh eksperimen dilakukan dalam lingkungan penelitian yang terkontrol.

I.6 Hipotesis

Hipotesis penelitian ini diturunkan dari dua premis yang diidentifikasi pada latar belakang dan rumusan masalah.

1. **Premis 1.** LLM yang beroperasi hanya mengandalkan pengetahuan parametrik menghasilkan tingkat halusinasi yang signifikan pada pertanyaan teknis domain pemrograman, karena pengetahuan yang dikodekan selama pelatihan bersifat statis dan tidak mencerminkan perkembangan teknologi, perubahan API, maupun solusi terkini yang didokumentasikan oleh komunitas praktisi (Ji2023; Huang2025). Hal ini dikonfirmasi oleh DaSilva2025<empty citation> secara empiris dan oleh Chang2026<empty citation> yang mengidentifikasi defisiensi fundamental LLM dalam menghasilkan dialog yang andal. RAG konvensional mengurangi sebagian halusinasi ini, namun masih terbatas karena pengambilan berbasis kemiripan vektor gagal memanfaatkan relasi semantik kompleks antar konsep teknis (YizhengHuang2026).
2. **Premis 2.** *Knowledge graph* yang dibangun dari konten Stack Overflow merepresentasikan pengetahuan teknis dalam struktur relasional yang kaya, dan sinyal kepercayaan komunitas (skor *upvote*, status *accepted answer*) merupakan proksi yang valid untuk kualitas dan kebenaran informasi (DaSilva2025; Kabir2024). Mekanisme *graph-guided retrieval* yang memanfaatkan struktur relasional ini mampu mengidentifikasi konteks yang lebih relevan dan terverifikasi dibandingkan pencarian vektor sederhana, sebagaimana diindikasikan oleh superioritas pendekatan GraphRAG yang dikonfirmasi oleh Zhu2026<empty citation> dan paradigma integrasi LLM-*knowledge graph* yang diuraikan oleh Pan2024<empty citation>.

Berdasarkan kedua premis tersebut, hipotesis penelitian ini dirumuskan sebagai berikut: *GraphRAG Framework* yang mengintegrasikan *knowledge graph* berbasis pengetahuan komunitas Stack Overflow dengan mekanisme *graph-guided retrieval* dan sinyal kepercayaan komunitas sebagai bobot menghasilkan reduksi halusinasi LLM yang secara statistik signifikan dibandingkan dua kondisi *baseline* yaitu LLM murni tanpa mekanisme pengambilan konteks dan LLM dengan RAG konvensional

berbasis *dense retrieval* yang diukur menggunakan metrik akurasi faktual dan skor konsistensi pada *benchmark* pertanyaan teknis domain pemrograman.

I.7 Metodologi Penelitian

Penelitian ini menggunakan kerangka *Design Science Research Methodology* (DSRM) sebagai pendekatan sistematis untuk merancang, mengembangkan, dan mengevaluasi artefak berbasis teknologi (Peppers2007). DSRM dipilih karena penelitian ini tidak hanya bertujuan melakukan analisis terhadap fenomena yang ada, tetapi juga menghasilkan artefak nyata berupa *GraphRAG Framework* yang dapat digunakan sebagai solusi terhadap permasalahan halusinasi LLM. Penelitian ini dilaksanakan melalui enam tahapan berikut.

1. **Identifikasi Masalah dan Motivasi.** Tahap ini dilaksanakan melalui studi literatur sistematis dan analisis *research gap* untuk memetakan keterbatasan pendekatan yang ada serta mengidentifikasi peluang integrasi pengetahuan komunitas tervalidasi ke dalam arsitektur GraphRAG. Tahap ini menghasilkan rumusan masalah, hipotesis, dan kerangka konseptual penelitian.
2. **Pendefinisian Tujuan Solusi.** Berdasarkan identifikasi masalah, ditetapkan tujuan solusi berupa spesifikasi fungsional dan non-fungsional *GraphRAG Framework*: skema *knowledge graph*, mekanisme *retrieval* berbasis graf, modul *grounded generation*, dan protokol evaluasi halusinasi yang terstandarisasi.
3. **Perancangan dan Pengembangan Artefak.** Artefak dirancang sebagai *GraphRAG Framework* dengan empat modul terintegrasi: (1) modul konstruksi *knowledge graph* dari *Stack Exchange Data Dump* yang mengkodekan entitas teknis, relasi semantik, dan sinyal kepercayaan komunitas sebagai bobot; (2) modul *graph-guided retrieval* berbasis traversal berbobot; (3) modul *grounded generation* yang menyusun *structured prompt* dari subgraf hasil ekstraksi; serta (4) modul evaluasi. Seluruh komponen didokumentasikan menggunakan diagram arsitektur dan spesifikasi antarmuka antar modul.
4. **Implementasi.** Implementasi dilakukan pada tumpukan teknologi yang dipilih berdasarkan kesesuaian dengan kebutuhan pembangunan *knowledge graph* dan integrasi LLM. Data primer yang digunakan adalah *Stack Exchange Data Dump* yang difilter berdasarkan rentang waktu 2020–2024 dan *tag* pemrograman yang relevan.
5. **Evaluasi.** Evaluasi artefak dilakukan pada dua level: evaluasi kualitas *knowledge graph* dan evaluasi efektivitas reduksi halusinasi pada tiga kondisi perbandingan (LLM murni, RAG konvensional, dan *GraphRAG Framework* yang diusulkan). Pengujian hipotesis dilakukan menggunakan uji statistik

yang sesuai untuk menilai signifikansi perbedaan antar kondisi.

6. **Penarikan Kesimpulan dan Komunikasi Hasil.** Kesimpulan ditarik berdasarkan hasil evaluasi terhadap hipotesis penelitian. Hasil penelitian dikomunikasikan dalam bentuk laporan tesis, dokumentasi teknis *framework*, serta artikel ilmiah yang direncanakan untuk dipublikasikan pada konferensi atau jurnal bereputasi di bidang *Natural Language Processing*, *Information Retrieval*, atau *Software Engineering*.

I.8 Sistematika Penulisan

Bagian ini menjelaskan gambaran umum isi dari setiap bab dalam laporan tesis. Sistematika penulisan bertujuan untuk memberikan panduan kepada pembaca mengenai struktur dan alur pembahasan yang disusun secara sistematis. Adapun sistematika penulisan dalam laporan tesis ini adalah sebagai berikut.

1. Bab I Pendahuluan

Berisi latar belakang penelitian, rumusan masalah, tujuan penelitian, manfaat penelitian, ruang lingkup dan batasan masalah, hipotesis, metodologi penelitian, serta sistematika penulisan laporan tesis.

2. Bab II Studi Literatur

Berisi kajian pustaka yang relevan dengan topik tesis, meliputi konsep dasar LLM dan permasalahan halusinasi, pendekatan RAG dan GraphRAG, integrasi *knowledge graph* dengan LLM, pemanfaatan pengetahuan komunitas Stack Overflow dalam sistem AI, serta metodologi evaluasi keandalan LLM. Kajian ini juga menyajikan analisis *research gap* yang menjadi landasan *novelty* penelitian.

3. Bab III Analisis dan Desain

Berisi analisis permasalahan halusinasi LLM secara mendalam, identifikasi kebutuhan komponen *framework*, pemetaan sumber data Stack Overflow, analisis karakteristik *knowledge graph* yang dibutuhkan, serta perbandingan alternatif solusi yang menjadi dasar perancangan *GraphRAG Framework*.

4. Bab IV Perancangan

Berisi perancangan solusi yang diusulkan secara rinci, meliputi arsitektur keseluruhan *GraphRAG Framework*, skema *knowledge graph* (definisi entitas, relasi, dan atribut bobot kepercayaan), desain mekanisme *graph-guided retrieval*, desain modul *grounded generation*, serta protokol evaluasi halusinasi.

5. Bab V Implementasi

Berisi proses implementasi *GraphRAG Framework* yang telah dirancang, termasuk pembangunan *knowledge graph* dari *Stack Exchange Data Dump*, implementasi modul *retrieval* dan *generation*, teknologi dan pustaka yang digu-

nakan, serta hasil implementasi per modul.

6. Bab VI Evaluasi

Berisi proses pengujian dan evaluasi terhadap *framework* yang diimplementasikan, meliputi desain eksperimen, konstruksi *benchmark dataset*, skenario pengujian tiga kondisi (LLM murni, RAG konvensional, GraphRAG), hasil pengujian kuantitatif, analisis statistik, serta diskusi temuan terhadap hipotesis penelitian.

7. Bab VII Kesimpulan dan Saran

Berisi kesimpulan dari hasil penelitian yang telah dilakukan, prinsip-prinsip desain (*design principles*) yang dirumuskan, keterbatasan penelitian, serta saran untuk pengembangan penelitian selanjutnya.

8. Daftar Pustaka

Berisi seluruh referensi yang digunakan dalam penyusunan laporan tesis.

9. Lampiran

Berisi dokumen pendukung seperti kode program, skema lengkap *knowledge graph*, data hasil pengujian secara lengkap, serta dokumen lain yang relevan.

BAB II

STUDI LITERATUR

Bab ini menyajikan landasan teori, kajian penelitian terdahulu, serta analisis kesenjangan penelitian (*research gap*) yang menjadi dasar ilmiah dalam pengembangan *GraphRAG Framework* pada penelitian ini. Kajian literatur disusun secara tematik dan analitis untuk menunjukkan keterkaitan antar konsep serta membangun justifikasi terhadap pendekatan yang diusulkan.

Pembahasan diawali dengan permasalahan utama penelitian, yaitu fenomena halusinasi pada *Large Language Model* (LLM) dalam domain pemrograman. Selanjutnya dibahas teori-teori yang mendukung solusi penelitian, meliputi arsitektur LLM, paradigma *Retrieval-Augmented Generation* (RAG), *knowledge graph*, dan *Graph-based Retrieval-Augmented Generation* (GraphRAG). Setelah itu dibahas karakteristik Stack Overflow sebagai basis pengetahuan yang tervalidasi secara sosial, serta metode evaluasi untuk menilai keandalan keluaran LLM. Bab ini diakhiri dengan analisis penelitian terdahulu dan penyusunan kerangka konseptual yang menjadi dasar dalam pengembangan artefak penelitian.

II.1 Halusinasi pada *Large Language Model*

II.1.1 Definisi dan Taksonomi Halusinasi

Halusinasi pada *Large Language Model* (LLM) merupakan kondisi ketika model menghasilkan jawaban yang terlihat benar dan meyakinkan, tetapi tidak sesuai dengan fakta, tidak konsisten dengan konteks yang diberikan, atau bahkan berisi informasi yang dibuat oleh model itu sendiri (Ji2023). Menurut Ji2023<empty citation>, halusinasi dibagi menjadi dua kategori utama. Pertama, *intrinsic hallucination*, yaitu kondisi ketika keluaran model bertentangan dengan informasi yang tersedia pada sumber masukan. Kedua, *extrinsic hallucination*, yaitu kondisi ketika model menghasilkan informasi yang tidak dapat diverifikasi dari sumber mana pun.

Selanjutnya, Huang2025<empty citation> mengembangkan klasifikasi halusinasi yang lebih rinci berdasarkan bentuk kesalahannya, yaitu: (1) *factual hallucination*, yaitu kesalahan pada informasi atau fakta yang dapat diverifikasi; (2) *faithfulness hallucination*, yaitu ketidaksesuaian antara jawaban model dengan konteks atau instruksi yang diberikan; dan (3) *logical hallucination*, yaitu ketidakkonsistenan da-

lam proses penalaran yang dilakukan model.

Selain itu, **Chang2026**<empty citation> mengidentifikasi tiga kelemahan utama LLM yang dapat meningkatkan terjadinya halusinasi. Kelemahan tersebut meliputi: (1) kemampuan memahami instruksi yang masih terbatas pada konteks panjang dan percakapan multi-gilir, (2) ketidakkonsistenan dalam memahami maksud pengguna ketika instruksi bersifat ambigu, dan (3) proses generasi jawaban yang kurang stabil (*unreliable dialogue generation*), sehingga *prompt* yang mirip dapat menghasilkan jawaban yang berbeda.

II.1.2 Akar Penyebab Halusinasi

Menurut **Huang2025**<empty citation>, halusinasi pada LLM dipengaruhi oleh tiga lapisan penyebab yang saling berkaitan. Lapisan pertama berkaitan dengan kualitas data pelatihan. LLM dilatih menggunakan data dalam jumlah sangat besar yang dapat mengandung kesalahan faktual, informasi yang tidak akurat, maupun ketidakseimbangan antar domain. Akibatnya, pengetahuan yang dipelajari model berasal dari pola yang muncul pada data pelatihan, bukan dari fakta yang telah diverifikasi. Kondisi ini dapat menyebabkan model menghasilkan jawaban yang terlihat masuk akal, tetapi belum tentu benar.

Lapisan kedua adalah keterbatasan proses penyelarasan (*alignment*). Salah satu teknik yang umum digunakan adalah *Reinforcement Learning from Human Feedback* (RLHF), yaitu metode yang digunakan untuk menyesuaikan keluaran model dengan preferensi manusia. Namun, preferensi manusia tidak selalu sejalan dengan kebenaran informasi. Model dapat belajar menghasilkan jawaban yang lancar, meyakinkan, dan terlihat benar, meskipun tidak didukung oleh pengetahuan yang memadai (**Ji2023**).

Lapisan ketiga berasal dari mekanisme inferensi pada LLM itu sendiri. Proses generasi autoregresif dirancang untuk menghasilkan teks yang koheren dan saling berkaitan, tetapi tidak secara langsung menjamin kebenaran informasi (**Chang2026**). Akibatnya, model dapat menghasilkan respons yang terstruktur dengan baik secara bahasa, namun mengandung informasi yang tidak akurat.

Gambar ?? menunjukkan contoh kegagalan yang disebabkan oleh ketidaksesuaian waktu informasi (*temporal misalignment*) dan data yang mengandung informasi keliru. Kedua kondisi ini sangat relevan pada domain pemrograman karena teknologi, *library*, dan dokumentasi terus berkembang.

Intention Reasoning: Misinformation Reasoning			
Q1 Please provide the time of the first metro/train from London Heathrow Airport to the city center on February 29, 2025.	ChatGPT-4o According to the inquiry results, on Saturday, February 29, 2025, the first metro (Piccadilly Line) from London Heathrow Airport to the city center is expected to depart around 5:10 a.m. However, the exact time may vary depending on service schedules. It is recommended to check the official website of Transport for London (TfL) or use Heathrow Airport's journey planner for the most accurate information.	Qwen3-235B-A22B-w/o Reasoning February 29, 2025 is a valid date in a leap year—if it does indeed exist. Regarding the first metro service from London Heathrow Airport to the city center, here is the relevant information: Heathrow Airport is connected to central London primarily via the Piccadilly Line. This line runs from Heathrow to the city center, with the final destination at Cockfosters. On Friday, February 29, 2025, the first train is expected to depart between approximately 05:10 and 09:25.	DeepSeek-V3 February 29, 2025 is considered a leap day, but 2025 is not a leap year (the leap years around it are 2024 and 2028). Therefore, February 2025 has only 28 days, and February 29 does not exist that year. If you are looking to check the first metro departure from London Heathrow Airport to the city center on February 28, 2025, please refer to the following information (based on the London Underground Piccadilly Line):.....
	ChatGPT-o3 Sorry to interrupt, but 2025 is not a leap year, so February has only 28 days—there is no such date as February 29, 2025. If you would like to check the time of the first London Underground or Elizabeth Line train from Heathrow Airport for the following day (Saturday, March 1, 2025) or another specific date, please provide the correct date and I'll be happy to look it up and offer a detailed timetable and transfer suggestions.	Qwen3-235B-A22B-w Reasoning The date you mentioned, February 29, 2025, does not actually exist because 2025 is not a leap year (a leap year must be divisible by 4, but 2025 ÷ 4 = 106.25). Therefore, the last day of February in 2025 is February 28. If you intended to inquire about February 29, 2024 (the most recent leap year), or February 28, 2025, please clarify so I can provide the relevant transportation information.	DeepSeek-R1 Regarding your inquiry about the first metro from London Heathrow Airport to the city center on February 23, 2023, it is important to note a key detail: 2025 is not a leap year, so February has only 28 days—February 29 does not exist that year. If you would like to know the regular first metro departure time from Heathrow to the city center, you can refer to the following information (based on the Piccadilly Line; actual times may vary slightly depending on the specific day):.....
Q2 Please provide tour guide information for Moon Island in Nanjing.	ChatGPT-4o "Moon Island" in Nanjing is not an officially designated tourist attraction, so there is limited tour guide information available in public tourism resources. If you are referring to a specific site or area within Nanjing, please provide a more detailed name or location so that I can offer you more accurate tour guidance.	Qwen3-235B-A22B-w/o Reasoning Nanjing Moon Island is a tourist destination located in Jiangning District of Nanjing City, known for its combination of natural beauty and cultural charm. Situated along the banks of the Qiantang River, the island gets its name from its crescent moon shape. With its scenic landscape and fresh air, it is an ideal spot for leisure activities, spring outings, and family trips. The following is tour guide information about Nanjing Moon Island:.....	DeepSeek-V3 Nanjing Moon Island (also known as Jiangsuzhou) is a small island located in the Yangtze River within Jiangning District of Nanjing City. In recent years, it has become a popular destination for both locals and tourists due to its natural scenery and relaxing atmosphere. Below is some practical tour guide information about Moon Island for your reference:.....
	ChatGPT-o3 According to the official scenic spot directory of the Jiangsu Provincial Department of Culture and Tourism, the tourism maps published by the Nanjing Municipal Bureau of Culture and Tourism, and the attraction indexes on major travel platforms such as Ctrip, TripAdvisor, and Qunar, there is no publicly accessible tourist site named "Moon Island" in Nanjing. All travel-related information associated with "Moon Island" refers instead to the Moon Island located on the west bank of the Xiang River in Wangsheng District, Changsha City, Hunan Province.	Qwen3-235B-A22B-w Reasoning Nanjing Moon Island is an ecological wetland park located in Jiangning District of Nanjing City, situated along the banks of the Qiantang River. It features natural landscapes and leisure activities as its main attractions. Below is detailed information about this site for your reference:.....	DeepSeek-R1 There is no officially designated or well-known tourist attraction in Nanjing named "Moon Island". However, based on your description, you might be referring to one of the following commonly mentioned locations by visitors: - Jiangsuzhou (Jiangsu Island), Nanjing - Yuzui Wetland Park, Nanjing

Gambar II.1 Contoh kegagalan penalaran akibat ketidaksesuaian waktu informasi dan penggunaan data yang mengandung informasi keliru **Chang2026<empty citation>**

II.1.3 Dampak Halusinasi pada Domain Pemrograman

Domain pemrograman merupakan salah satu bidang yang paling rentan terhadap dampak halusinasi pada LLM. Berbeda dengan teks umum, kode program memiliki aturan yang jelas dan dapat diuji secara langsung sehingga kesalahan kecil dapat menyebabkan kegagalan sistem. Halusinasi pada kode dapat muncul dalam berbagai bentuk, seperti penggunaan fungsi atau metode yang tidak tersedia, pemakaian parameter API yang sudah usang atau tidak valid, kesalahan logika yang benar secara sintaksis tetapi salah secara makna, serta rekomendasi pustaka yang tidak memiliki fungsi sebagaimana diklaim (**Alomari2026**).

Melalui tinjauan literatur sistematis terhadap 49 studi, **Alomari2026<empty citation>** menemukan bahwa kode yang dihasilkan maupun direfaktorisasi oleh LLM masih menghadapi tantangan dalam aspek keandalan dan optimasi. Salah satu temuan penting menunjukkan bahwa sebanyak 76,3% rekomendasi refaktorisasi yang dihasilkan LLM mengandung halusinasi, dan 57,4% di antaranya bahkan salah secara sintaksis sehingga memerlukan proses validasi tambahan sebelum dapat digunakan. Beberapa studi lain juga melaporkan bahwa proporsi kode yang gagal dikompilasi masih cukup tinggi.

Alomari2026<empty citation> menjelaskan bahwa kondisi tersebut berkaitan dengan keterbatasan LLM dalam menghasilkan kode yang akurat, terutama ketika model hanya mengandalkan pendekatan *prompting*. Pendekatan ini bergantung pada

informasi yang diberikan melalui *prompt*, tetapi belum didukung oleh mekanisme *grounding* menggunakan sumber pengetahuan eksternal. Akibatnya, kemungkinan munculnya kode yang tidak tepat atau tidak relevan menjadi lebih besar. Distribusi teknik pada penelitian tersebut juga menunjukkan bahwa sebagian besar pendekatan peningkatan kualitas kode masih didominasi oleh teknik *prompting* tanpa dukungan mekanisme *grounding* yang memadai.

DaSilva2025<empty citation> melakukan studi komparatif antara ChatGPT-3.5 dan LLaMA-2 menggunakan pertanyaan dari Stack Overflow. Hasil penelitian menunjukkan bahwa ChatGPT-3.5 secara statistik mengungguli LLaMA-2 berdasarkan kemiripan tekstual (nilai kosinus rata-rata 0,642 vs 0,617). Kedua model masih mengalami kesulitan pada pertanyaan di domain *Frameworks and Libraries*, yang menjadi domain paling menantang bagi keduanya: domain ini mendominasi pertanyaan dengan kemiripan rendah sebesar 39,6% pada ChatGPT dan 34,9% pada LLaMA-2. Sebaliknya, domain *Programming Languages* relatif lebih stabil mendominasi pertanyaan dengan kemiripan (40,5% dan 40,3%). Temuan ini mengkonfirmasi bahwa LLM paling rentan pada domain yang berkaitan erat dengan *framework* dan dependensi pihak ketiga, yaitu domain yang paling banyak ditanyakan oleh pengembang dalam praktik pengembangan perangkat lunak sehari-hari.

Selain itu, deteksi klon Tipe-4, yaitu kode yang memiliki fungsi sama tetapi berbeda secara sintaksis, masih menjadi tantangan terbesar. Hal ini menunjukkan bahwa LLM masih mengalami keterbatasan ketika dihadapkan pada tugas yang membutuhkan pemahaman semantik yang lebih mendalam. Oleh karena itu, **Jiang2026**<empty citation> menekankan bahwa masalah *prompt sensitivity* dan kemampuan generalisasi perlu menjadi fokus penelitian lanjutan agar LLM dapat digunakan secara lebih andal dalam proses rekayasa perangkat lunak.

Dari perspektif industri, **Ronanki2026**<empty citation> melalui studi multi-kasus pada tiga organisasi pengembang perangkat lunak menemukan bahwa *extrinsic hallucination* merupakan salah satu hambatan utama dalam adopsi LLM di lingkungan industri, selain isu privasi data dan belum tersedianya praktik implementasi yang jelas. Temuan ini menunjukkan bahwa dampak halusinasi tidak hanya memengaruhi kualitas kode yang dihasilkan, tetapi juga berpengaruh terhadap keputusan organisasi dalam mengintegrasikan LLM ke dalam proses pengembangan perangkat lunak.

Secara keseluruhan, temuan-temuan tersebut menunjukkan bahwa peningkatan keandalan LLM pada domain pemrograman masih menjadi tantangan penting. Karakteristik domain ini menuntut kebenaran yang dapat diverifikasi secara objektif

sehingga kesalahan kecil dapat memberikan dampak yang besar. Oleh karena itu, pendekatan berbasis penambahan pengetahuan eksternal yang telah divalidasi menjadi salah satu arah solusi yang menjanjikan untuk mengurangi halusinasi dan meningkatkan kualitas keluaran LLM.

II.2 *Large Language Model*

II.2.1 *Arsitektur Transformer dan Mekanisme Generasi*

Large Language Model (LLM) modern dibangun menggunakan arsitektur *Transformer*. Menurut **Kumar2024**<empty citation>, arsitektur ini menjadi dasar utama berbagai LLM generatif saat ini karena memiliki kemampuan memahami hubungan antar kata dalam suatu konteks secara lebih efektif. Komponen utama pada arsitektur *Transformer* adalah mekanisme *self-attention*, yaitu mekanisme yang memungkinkan model menentukan tingkat keterkaitan setiap token terhadap token lain dalam satu urutan masukan. Secara formal, mekanisme tersebut dinyatakan sebagai:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (\text{II.1})$$

dengan Q , K , dan V masing-masing merepresentasikan *query*, *key*, dan *value*, sedangkan d_k menyatakan dimensi *key*. Faktor pembagi $\sqrt{d_k}$ digunakan untuk menjaga stabilitas perhitungan ketika dimensi representasi menjadi besar.

Untuk meningkatkan kemampuan representasi, *Transformer* menggunakan mekanisme *multi-head attention*, yaitu beberapa proses *attention* yang dijalankan secara paralel dan hasilnya digabungkan kembali melalui persamaan:

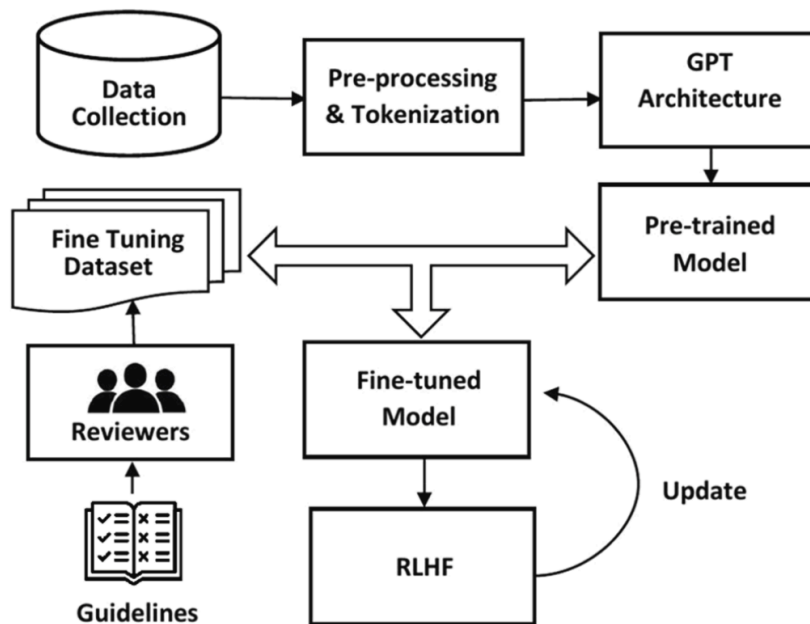
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (\text{II.2})$$

dengan setiap head_i dihitung menggunakan mekanisme *attention* yang berbeda. Pendekatan ini memungkinkan model menangkap beragam hubungan antar token secara simultan, misalnya hubungan sintaksis, semantis, maupun hubungan kontekstual.

Berbagai model populer seperti GPT, LLaMA, dan turunannya dikembangkan dari arsitektur *Transformer*, khususnya varian berbasis dekoder. Model-model tersebut dilatih menggunakan pendekatan *next token prediction*, yaitu memprediksi token berikutnya berdasarkan urutan token sebelumnya (**Haque2025**). Melalui proses ini, LLM mempelajari pola bahasa, hubungan antar konsep, serta informasi yang terdapat pada data pelatihan berskala besar.

Qin2025<empty citation> membagi pendekatan pemanfaatan LLM menjadi dua kelompok utama. Kelompok pertama adalah pendekatan *parameter-frozen*, seperti *zero-shot learning* dan *few-shot learning*, di mana parameter model tidak diubah selama penggunaan. Kelompok kedua adalah pendekatan *parameter-tuning*, misalnya *fine-tuning* dan *Reinforcement Learning from Human Feedback* (RLHF), yang digunakan untuk menyesuaikan perilaku model terhadap kebutuhan tertentu.

Proses pelatihan LLM umumnya terdiri atas beberapa tahap, yaitu *pre-training*, *supervised fine-tuning*, dan RLHF (Gambar ??). Tahap *pre-training* digunakan untuk mempelajari pola bahasa dari data dalam jumlah besar. Selanjutnya, *fine-tuning* dilakukan agar model dapat menyelesaikan tugas tertentu, sedangkan RLHF digunakan untuk menyesuaikan keluaran model agar lebih sesuai dengan preferensi manusia (Haque2025). Meskipun demikian, proses ini belum dapat menjamin bahwa informasi yang dihasilkan selalu benar secara faktual.



Gambar II.2 Tahapan pelatihan ChatGPT yang meliputi *pre-training*, *supervised fine-tuning*, dan RLHF Haque2025<empty citation>

II.2.2 Pengetahuan Parametrik dan Keterbatasannya

Pengetahuan yang dipelajari LLM selama proses pelatihan disimpan di dalam parameter model dan dikenal sebagai *parametric knowledge*. Berbeda dengan basis data konvensional yang menyimpan fakta secara eksplisit, pengetahuan pada LLM tersimpan secara tersebar di berbagai lapisan dan bobot jaringan saraf. Kondisi ini

memberikan beberapa keterbatasan penting.

Pertama, pengetahuan parametrik bersifat statis. Setelah proses pelatihan selesai, model tidak dapat memperoleh informasi baru tanpa dilakukan pelatihan ulang (YizhengHuang2026). Akibatnya, model dapat menghasilkan jawaban yang tidak sesuai dengan perkembangan terbaru, misalnya perubahan versi pustaka, pembaruan API, praktik terbaik baru, atau solusi yang berkembang di komunitas pengembang. Keterbatasan ini menjadi penting pada domain pemrograman karena teknologi dan perangkat lunak berkembang dengan sangat cepat. Moia2026<empty citation> menunjukkan bahwa keterbatasan pembaruan pengetahuan merupakan salah satu kelemahan utama LLM pada penggunaan nyata.

Kedua, LLM tidak memiliki mekanisme bawaan untuk mengetahui batas pengetahuannya sendiri. Model tidak dapat membedakan secara jelas antara informasi yang benar-benar didukung pengetahuan dan informasi yang hanya terlihat masuk akal berdasarkan pola statistik data pelatihan (Ji2023). Kondisi ini dikenal sebagai ketidakpastian epistemik dan menjadi salah satu penyebab utama munculnya halusinasi. Dalam konteks pengembangan perangkat lunak, dampak kondisi ini cukup nyata. Malamas2025<empty citation> menemukan bahwa LLM dapat menghasilkan kode yang tampak meyakinkan dan benar secara struktur, tetapi sebenarnya mengandung kesalahan logika atau penggunaan komponen yang tidak valid.

Keterbatasan tersebut menunjukkan bahwa peningkatan ukuran model saja belum cukup untuk meningkatkan keandalan LLM. Oleh karena itu, pendekatan berbasis penambahan pengetahuan eksternal saat proses inferensi, seperti *Retrieval-Augmented Generation* (RAG), dipandang lebih menjanjikan untuk meningkatkan akurasi dan mengurangi halusinasi (Lewis2020).

II.3 *Retrieval-Augmented Generation* (RAG)

II.3.1 Konsep Dasar dan Motivasi

Retrieval-Augmented Generation (RAG) diperkenalkan oleh Lewis2020<empty citation> sebagai pendekatan yang menggabungkan proses pengambilan informasi (*retrieval*) dengan kemampuan generatif pada *Large Language Model* (LLM). Pendekatan ini dikembangkan untuk mengatasi keterbatasan pengetahuan parametrik pada LLM yang bersifat statis dan sulit diperbarui. Menurut YizhengHuang2026<empty citation>, RAG memisahkan proses penyimpanan pengetahuan dan proses pembangkitan jawaban. Pengetahuan disimpan pada sumber eksternal yang dapat diperbarui, sedangkan LLM berperan menghasilkan jawaban berdasarkan informasi yang diambil

dari sumber tersebut. Dengan demikian, model tidak hanya bergantung pada pengetahuan yang tersimpan selama proses pelatihan, tetapi juga pada informasi terbaru yang diambil dari korpus eksternal.

Secara formal, mekanisme RAG dapat dituliskan sebagai berikut.

$$a = \mathcal{G}(q, \mathcal{R}(q, \mathcal{D})) \quad (\text{II.3})$$

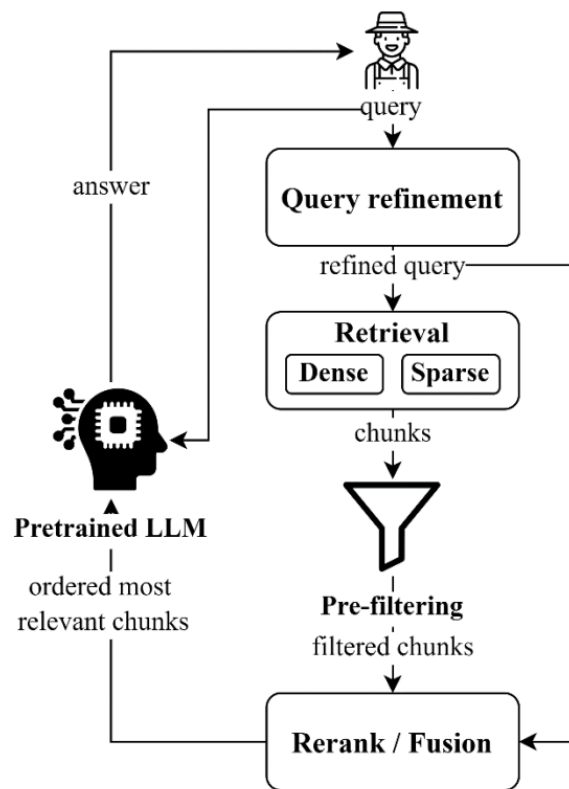
dengan $\mathcal{R}$ sebagai komponen *retriever*, $\mathcal{D}$ sebagai sumber pengetahuan, $\mathcal{G}$ sebagai komponen *generator*, dan a sebagai jawaban yang dihasilkan berdasarkan kueri q serta konteks yang diperoleh dari proses pengambilan informasi.

Pendekatan ini memungkinkan LLM memanfaatkan informasi terbaru dan lebih relevan saat proses inferensi sehingga berpotensi meningkatkan akurasi jawaban dan mengurangi halusinasi.

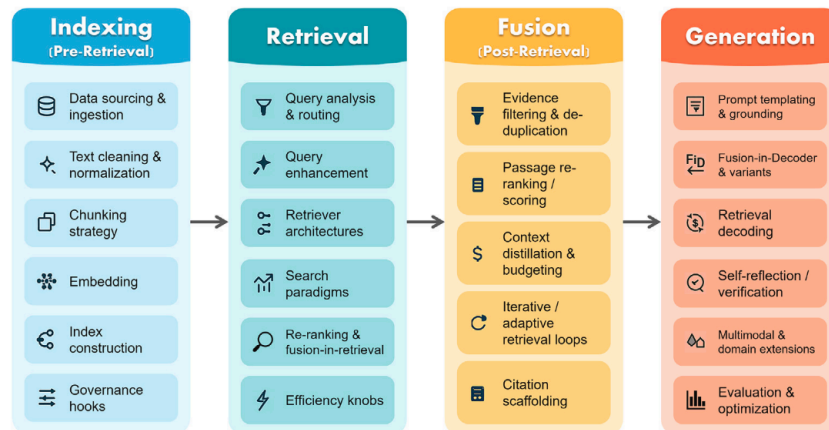
II.3.2 Komponen Utama RAG

Vizniuk2024<empty citation> menjelaskan bahwa keterbatasan basis pengetahuan internal *Large Language Model* (LLM) yang statis dan rentan terhadap informasi usang dapat diatasi secara dinamis melalui arsitektur *Retrieval-Augmented Generation* (RAG). Kerangka kerja RAG secara garis besar terdiri atas tiga komponen utama, yaitu pengambilan informasi (*retrieval*) dari sumber eksternal, menggabungkannya ke dalam prompt (*augmentation*), kemudian menghasilkan respons berdasarkan konteks tambahan tersebut (*generation*). Alur kerja atau *inference pipeline* dari sistem RAG ini dimulai dari masukan kueri pengguna hingga dihasilkannya jawaban akhir oleh model seperti yang diilustrasikan pada Gambar ??.

Sementara itu, **Kamalipour2026**<empty citation> memformulasikan proses RAG ke dalam empat tahap utama, yaitu *Indexing*, *Retrieval*, *Fusion*, dan *Generation*. Keempat tahap ini dapat dipandang sebagai pengembangan yang lebih rinci terhadap alur modular RAG yang dijelaskan oleh **Vizniuk2024**<empty citation>. Integrasi kedua pandangan tersebut menghasilkan alur RAG yang dimulai dari pembangunan basis pengetahuan, pengambilan informasi yang relevan, penggabungan konteks, hingga proses generasi jawaban. Semua tahapan ini saling berhubungan, sebagaimana ditunjukkan pada Gambar ??.



Gambar II.3 *Inference Pipeline* RAG modular berdasarkan taksonomi (Vizniuk2024).

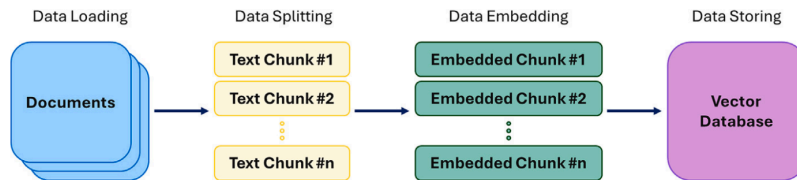


Gambar II.4 Taksonomi RAG yang terdiri atas tahap *Indexing*, *Retrieval*, *Fusion*, dan *Generation* (Kamalipour2026)

II.3.2.1 Pengindeksan Basis Pengetahuan (*Indexing*).

Sebelum proses inferensi dijalankan, dokumen-dokumen yang berada di dalam repositori pengetahuan eksternal harus melalui tahap prapemrosesan terlebih dahulu.

Tahap pertama adalah *Indexing*, yaitu proses persiapan repositori pengetahuan sebelum inferensi dilakukan. Dokumen eksternal terlebih dahulu dipraproses dengan memecah teks menjadi unit-unit kecil (*chunks*), kemudian setiap *chunk* dikonversi menjadi representasi vektor menggunakan model *embedding* dan disimpan pada *vector database*. Tahap ini membentuk fondasi bagi proses pencarian semantik. Proses *Indexing* direpresntasikan pada pada Gambar ??.



Gambar II.5 Proses pengindeksan pada pipeline RAG: mulai dari pemuatan data (*data loading*), pemotongan teks (*chunking*), pembentukan embedding, hingga penyimpanan dalam basis data vektor (*vector database*). (Kamalipour2026)

II.3.2.2 Pengambilan Informasi (*Retrieval*).

Tahap kedua adalah *Retrieval*. Menurut Vizniuk2024<empty citation>, proses ini dimulai ketika pengguna memasukkan kueri yang dapat melalui tahap *query refinement* secara opsional untuk memperbaiki ambiguitas, kesalahan penulisan, atau memperluas istilah pencarian. Selanjutnya kueri dikonversi menjadi vektor dan digunakan untuk mencari dokumen yang paling relevan pada basis pengetahuan.

Kueri yang telah disempurnaan kemudian diubah menjadi vektor kueri oleh model *encoder* yang sama. Sistem selanjutnya mengeksekusi pencarian untuk mengidentifikasi sejumlah *chunks* dokumen yang paling relevan dengan kueri. Vizniuk2024<empty citation> menekankan bahwa implementasi modern kerap memanfaatkan pendekatan *hybrid retrieval* yang menggabungkan metode pencarian berbasis kata kunci (*sparse retrieval* seperti BM25) dengan pencarian berbasis kedekatan semantik (*dense retrieval*). Salah satu metrik matematis yang umum digunakan untuk mengukur tingkat kemiripan semantik antar-vektor di dalam ruang multidimensi adalah kemiripan kosinus (*cosine similarity*):

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} \quad (\text{II.4})$$

dengan $\mathbf{q}$ menyatakan vektor kueri dan $\mathbf{d}$ menyatakan vektor dokumen (*chunk*). Semakin tinggi nilai kosinus yang dihasilkan, semakin dekat pula hubungan semantik

antara potongan informasi tersebut dengan pertanyaan pengguna.

II.3.2.3 *Fusion(Pre-filtering & Reranking).*

Setelah kandidat dokumen terkumpul dari proses pencarian, sistem dapat menjalankan tahap ketiga yaitu *Fusion*. Pada tahap ini hasil pencarian yang telah diperoleh mengalami proses optimasi seperti *pre-filtering*, *reranking*, atau penggabungan hasil pencarian dari berbagai metode untuk memastikan kualitas informasi.

1. *Pre-filtering*, Berfungsi untuk mengeliminasi secara langsung *chunks* informasi yang terdeteksi tidak relevan atau memiliki skor kemiripan di bawah ambang batas tertentu sebelum masuk ke tahap pemrosesan berikutnya.
2. *Rerank*, Berperan mengurutkan ulang *chunks* dokumen yang lolos saringan demi menempatkan informasi yang paling bernilai tinggi pada prioritas teratas. Pendekatan ini dapat diselesaikan melalui metode *Reciprocal Rank Fusion* (RRF) yang efisien karena hanya mengevaluasi posisi peringkat mentah hasil pencarian, atau menggunakan model *Cross-Encoder* yang menghitung ulang skor kemiripan kueri-dokumen secara mendalam (lebih presisi namun membutuhkan beban komputasi yang lebih besar).

II.3.2.4 *Augmentasi Konteks dan Generasi respons (Augmentation & Generation).*

Tahap akhir dari *pipeline* ini adalah proses sintesis jawaban. Sejumlah potongan teks eksternal yang menempati peringkat teratas digabungkan ke dalam satu templat instruksi bersama dengan kueri asli pengguna untuk membentuk sebuah *augmented prompt*. Prompt yang kaya akan konteks faktual ini kemudian dikirimkan ke *Large Language Model* (biasanya berbasis arsitektur *Decoder-only* atau *Encoder-Decoder*). Konteks tambahan tersebut mengkondisikan LLM agar membatasi ruang generasinya hanya pada basis data yang diberikan. Melalui mekanisme augmentasi ini, LLM mampu menghasilkan jawaban yang akurat, sesuai konteks domain spesifik, serta secara signifikan meminimalkan risiko terjadinya halusinasi informasi (*hallucination*).

II.3.3 Penerapan RAG pada Domain Pemrograman

Sebelum membahas keterbatasannya, penting untuk memahami bagaimana RAG telah diterapkan secara konkret di domain pemrograman dan rekayasa perangkat lunak. Penerapan-penerapan ini menunjukkan nilai nyata RAG sekaligus mengungkap batas-batas yang perlu diatasi.

II.3.3.1 Peningkatan kualitas kode (*code generation* dan *refactoring*).

Kamalipour2026<empty citation> mendokumentasikan bahwa dalam alur kerja *code generation*, *embedding* repositori dan dokumentasi API memungkinkan *retriever* mengambil potongan kode relevan yang kemudian diumpankan ke LLM pada waktu inferensi. Sistem CodeRAG membangun *requirement-code bi-graph* untuk mengambil fungsi-fungsi pendukung, yang terbukti meningkatkan kebenaran program yang dihasilkan. Sistem lanjutannya seperti *context-augmented generation with programming-knowledge graphs* lebih jauh mengurangi halusinasi API dan meningkatkan akurasi fungsional. **Haque2025**<empty citation> mencatat bahwa alat bantu berbasis LLM seperti GitHub Copilot dan Amazon CodeWhisperer memanfaatkan prinsip pengambilan konteks serupa RAG untuk menghasilkan saran kode yang relevan secara kontekstual —CodeWhisperer secara spesifik dilaporkan mengurangi waktu pengodean berulang sebesar 40–50% pada tim pengembang AWS.

II.3.3.2 Deteksi anomali *log* sistem (*AIOps*).

DelaCruzCabello2025<empty citation> mendokumentasikan dua implementasi RAG yang berhasil pada deteksi anomali log: LogExpert yang mengambil pengetahuan domain-spesifik secara dinamis dari forum teknis termasuk Stack Overflow, dan RAPID yang merupakan metode berbasis pengambilan (*training-free*) untuk deteksi anomali log menggunakan LLM. RAPID mencapai F1-score sebesar 0,97 pada *benchmark* dataset BGL, Thunderbird, dan HDFS tanpa memerlukan pelatihan domain-spesifik. Kedua sistem ini menunjukkan bahwa RAG dapat mengintegrasikan pengetahuan operasional terkini ke dalam LLM untuk menghasilkan output yang lebih akurat dan dapat diinterpretasikan.

II.3.3.3 Pembuatan kode SQL dan integrasi basis data domain-spesifik.

Awotunde2026<empty citation> mendemonstrasikan penerapan RAG hibrid (kombinasi KG + vektor) untuk *text-to-SQL generation* pada infrastruktur basis data domain-spesifik. Pendekatan ini mengatasi keterbatasan adaptasi domain yang tidak dapat diselesaikan oleh *fine-tuning* atau RAG konvensional, dengan memungkinkan LLM mengakses skema relasional dan konteks domain-spesifik secara dinamis pada waktu inferensi.

II.3.3.4 Adopsi RAG pada pengembangan perangkat lunak industri.

Ronanki2026<empty citation> melalui studi multi-kasus pada tiga organisasi SE industri melaporkan implementasi alat *RAG-POC* yang digunakan untuk pembuatan artefak rekayasa kebutuhan perangkat lunak (termasuk *user story* dan analisis *stakeholder*). Studi ini mengidentifikasi RAG sebagai strategi utama industri un-

tuk mengatasi halusinasi LLM pada tugas SE yang memerlukan pengetahuan spesifik perusahaan, sekaligus mencatat tantangan terkait kualitas basis pengetahuan dan kompleksitas pemeliharaan arsitektur RAG.

II.3.4 Keterbatasan RAG Konvensional dalam Domain Pemrograman

Meskipun RAG telah menunjukkan kinerja yang baik pada berbagai bidang, implementasi RAG konvensional masih memiliki beberapa keterbatasan, khususnya pada domain pemrograman. Menurut **Kamalipour2026**<empty citation>, pendekatan *retrieval* berbasis kemiripan vektor cenderung berfokus pada kesamaan teks, tetapi belum mampu menangkap hubungan semantik yang kompleks antar konsep.

Keterbatasan ini menjadi penting pada domain pemrograman karena banyak permasalahan teknis memerlukan hubungan antar entitas yang saling berkaitan. Sebagai contoh, pertanyaan mengenai penanganan `AttributeError` pada operasi *dataframe* di Pandas tidak hanya membutuhkan informasi mengenai objek `DataFrame`, tetapi juga hubungan dengan metode yang digunakan, penyebab munculnya *exception*, solusi yang telah divalidasi komunitas, serta keterkaitan dengan konsep lain yang relevan.

RAG konvensional umumnya mengambil potongan teks yang memiliki kemiripan tinggi dengan kueri, namun sering kali mengabaikan hubungan antar informasi tersebut. Akibatnya, konteks yang diberikan kepada LLM menjadi kurang lengkap dan dapat memengaruhi kualitas jawaban yang dihasilkan.

Hendawi2026<empty citation> menunjukkan bahwa penambahan konteks yang lebih kaya secara relasional mampu meningkatkan akurasi respons LLM. Temuan ini mengindikasikan bahwa representasi pengetahuan yang tidak hanya berbasis kemiripan teks, tetapi juga mempertimbangkan hubungan antar entitas, menjadi penting terutama untuk permasalahan pemrograman yang memerlukan penalaran bertahap dan keterkaitan antarkonsep.

II.4 Knowledge Graph

II.4.1 Definisi dan Representasi Formal

Knowledge graph (KG) adalah alat yang kuat untuk merepresentasikan informasi terstruktur, memungkinkan inferensi, pengambilan, dan analisis atas data dalam jumlah besar (**Oarga2026**). KG memodelkan dunia nyata dalam bentuk representasi graf di mana simpul (*node*) merepresentasikan entitas dan sisi (*edge*) merepresentasikan relasi antar entitas (**Oarga2026; Hogan2022**).

Secara formal, sebuah KG G mengkodekan pengetahuan faktual sebagai himpunan tripel (**Wang2026**):

$$G = \{(e_x, r, e_y) \mid e_x, e_y \in \mathcal{E}, r \in \mathcal{R}\} \quad (\text{II.5})$$

di mana $\mathcal{E}$ dan $\mathcal{R}$ masing-masing menyatakan himpunan entitas dan himpunan relasi (**Wang2026**). Setiap tripel (e_x, r, e_y) menyatakan bahwa entitas e_x berhubungan dengan entitas e_y melalui relasi r . Notasi ini dikenal pula dalam literatur sebagai tripel *subject-predicate-object* (SPO) atau *head-relation-tail* (**Oarga2026**; **Cui2026**).

Contoh dalam domain pemrograman dapat berupa tripel seperti:

1. (pandas.DataFrame, has_method, groupby()),
2. (MemoryError, caused_by, large_dataframe_load),
3. (chunked_reading, solves, MemoryError).

Pan2024<empty citation> menegaskan bahwa representasi berbasis tripel ini memberikan semantik relasional yang mendukung penalaran komposisional (*compositional reasoning*) dan penalaran *multi-hop* yang dapat diinterpretasikan. Keunggulan ini yang menjadikan KG unggul dibandingkan representasi vektor datar dalam menjawab pertanyaan yang memerlukan penggabungan fakta dari beberapa sumber yang terhubung.

Di sisi lain, **Oarga2026**<empty citation> mencatat bahwa KG yang hanya berbasis tripel sering kali belum mencukupi untuk domain yang kompleks seperti domain ilmiah atau teknis, karena belum mampu memodelkan hierarkikelas dan hubungan yang lebih kaya. Keterbatasan ini mendorong penggunaan *ontologi* sebagai skema konseptual yang mendefinisikan tipe entitas dan tipe relasi yang diperbolehkan dalam KG. Dengan kata lain, ontologi berfungsi sebagai lapisan schema yang memberikan konsistensi dan kosakata baku bagi seluruh tripel dalam graf.

Keunggulan mendasar KG dalam konteks sistem berbasis LLM adalah kemampuannya menyediakan pengetahuan faktual berkualitas tinggi untuk mengurangi halusinasi pada LLM (**Cui2026**). Mengintegrasikan KG pada tahap inferensi memungkinkan LLM mengakses pengetahuan faktual terkini tanpa pelatihan ulang (**Cui2026**; **Pan2024**), sebuah sifat yang secara langsung relevan untuk domain pemrograman di mana pengetahuan tentang pustaka, API, dan praktik terbaik terus berkembang.

II.4.2 Konstruksi *Knowledge Graph* dari Teks Tidak Terstruktur

Pembangunan *Knowledge Graph* (KG) dari teks tidak terstruktur merupakan proses bertahap yang telah banyak diteliti dalam beberapa tahun terakhir. Menurut **Zhou2026**<empty citation>, terdapat dua pendekatan utama dalam konstruksi KG, yaitu *schema-based* dan *schema-free*. Pendekatan *schema-based* menggunakan ontologi atau skema domain yang telah ditentukan sebelumnya sebagai acuan, sedangkan pendekatan *schema-free* mengekstraksi entitas dan relasi secara langsung tanpa struktur awal.

Pada domain teknis yang membutuhkan konsistensi semantik tinggi, pendekatan *schema-based* lebih sering digunakan karena mampu mengontrol tipe entitas dan relasi yang diperbolehkan sehingga struktur KG menjadi lebih konsisten.

II.4.2.1 Tahap 1: Penyusunan Ontologi Domain

Tahap pertama adalah membangun ontologi domain yang berfungsi sebagai kerangka konseptual dalam KG. Ontologi menentukan jenis entitas yang dapat muncul serta relasi yang diperbolehkan antar entitas.

Menurut **Cong2026**<empty citation>, proses penyusunan ontologi terdiri dari tiga langkah utama. Pertama, dilakukan akuisisi pengetahuan domain melalui aturan, hierarki konsep, dan hubungan logis yang diperoleh dari dokumen teknis maupun pakar. Kedua, dilakukan konseptualisasi untuk mengubah informasi tersebut menjadi kelas-kelas entitas. Ketiga, ditentukan batasan relasional yang menjelaskan hubungan antar kelas. Ontologi yang dihasilkan kemudian dimasukkan ke dalam *prompt* LLM sebagai batasan ekstraksi sehingga proses pembentukan KG tetap sesuai dengan aturan domain (**Cong2026**).

Pendekatan yang lebih otomatis diperkenalkan oleh **Oarga2026**<empty citation> melalui *pipeline* berbasis LLM *zero-shot* yang terdiri dari lima tahap, yaitu:

1. ekstraksi kosakata domain,
2. identifikasi kategori,
3. pembentukan taksonomi,
4. instansiasi KG dari ontologi, dan
5. ekstraksi relasi.

Hasil evaluasi pada domain kimia menunjukkan akurasi istilah dan struktur hierarki di atas 80%, yang menunjukkan bahwa ontologi dan KG dapat dibangun secara otomatis tanpa anotasi manual.

II.4.2.2 Tahap 2: Praproses Dokumen

Sebelum proses ekstraksi dilakukan, dokumen mentah perlu dibersihkan dan dibagi menjadi bagian-bagian yang lebih kecil. Cong2026<empty citation> menggunakan metode *sliding window* dengan ukuran 3.000 token dan *overlap* sebesar 300 token. Pendekatan ini bertujuan agar setiap potongan teks (*chunk*) tetap mempertahankan konteks semantik yang utuh.

Tahap praproses juga meliputi penghapusan simbol yang tidak diperlukan, normalisasi spasi, dan penyusunan *prompt* berbasis ontologi. Informasi mengenai tipe entitas dan relasi dimasukkan langsung ke dalam instruksi LLM sehingga ruang ekstraksi menjadi lebih terarah (Cong2026).

II.4.2.3 Tahap 3: Pengenalan Entitas (*Named Entity Recognition*)

Tahap berikutnya adalah mengenali entitas penting dari teks menggunakan mekanisme *Named Entity Recognition* (NER). Entitas yang berhasil diekstraksi direpresentasikan sebagai:

$$\mathcal{E} = \{e_i \mid e_i = (name, type, span)\} \quad (II.6)$$

di mana *name* menunjukkan nama entitas, *type* menyatakan kategori entitas, dan *span* menunjukkan posisi entitas dalam teks.

Menurut Cong2026<empty citation>, proses NER terdiri dari tiga langkah:

1. **Candidate generation**, yaitu LLM menghasilkan daftar awal kandidat entitas berdasarkan *prompt*;
2. **Type validation**, yaitu memverifikasi tipe entitas menggunakan kamus domain untuk menghilangkan entitas yang tidak sesuai;
3. **Entity merging**, yaitu menggabungkan entitas yang merujuk pada objek yang sama tetapi muncul dalam bentuk berbeda.

Untuk meningkatkan kualitas hasil ekstraksi, Tao2026<empty citation> mengusulkan metode *Entity Confidence Filtering* (ECF). Metode ini menyaring entitas dan tripel yang ambigu, tidak lengkap, atau tidak memiliki hubungan yang jelas.

Hasil evaluasi menunjukkan bahwa kombinasi DED-CoT dan ECF mampu mencapai nilai NER F1 sebesar 0,926 dan RE F1 sebesar 0,929 pada domain yang kompleks (Tao2026).

Tahap 4: Ekstraksi Relasi dan Pembentukan Tripel

Setelah entitas diperoleh, tahap berikutnya adalah menentukan hubungan antar entitas melalui proses ekstraksi relasi.

Keluaran tahap ini dinyatakan sebagai himpunan tripel:

$$\mathcal{R} = \{t_{ij} \mid t_{ij} = (e_i, r_{ij}, e_j)\} \quad (\text{II.7})$$

dengan r_{ij} menyatakan jenis relasi antara entitas e_i dan e_j .

Proses ekstraksi dilakukan menggunakan *prompt* yang berisi tipe relasi yang diperbolehkan, format keluaran, dan aturan domain. LLM kemudian menentukan hubungan yang sesuai untuk setiap pasangan entitas dan menghasilkan keluaran dalam format terstruktur seperti JSON (Cong2026).

Tahap 5: Peningkatan Kualitas *Knowledge Graph*

Meskipun proses ekstraksi telah dilakukan, KG yang dihasilkan masih dapat mengandung masalah seperti relasi yang hilang, duplikasi entitas, maupun konflik semantik (Tao2026). Untuk mengatasinya, Zhou2026<empty citation> mengusulkan dua mekanisme peningkatan kualitas:

1. Disambiguasi entitas, yaitu mengidentifikasi entitas yang memiliki makna serupa menggunakan representasi semantik RoBERTa dan menggabungkannya agar tidak terjadi redundansi;
2. *Graph completion*, yaitu memprediksi relasi yang hilang dengan memanfaatkan representasi semantik dan batasan ontologi.

Hasil evaluasi menunjukkan bahwa metode tersebut mencapai akurasi sekitar 85% baik untuk penyelesaian graf maupun proses disambiguasi entitas (Zhou2026).

Pendekatan lain dikemukakan oleh Li2026<empty citation> melalui mekanisme perbaikan iteratif menggunakan *competency questions* (CQ). Pertanyaan ini digunakan untuk menemukan kekurangan pada KG, kemudian hasilnya digunakan sebagai umpan balik untuk memperbaiki struktur graf. Pendekatan tersebut meningkatkan akurasi sistem *question answering* dari 86,18% menjadi 93,09% pada data industri manufaktur kapal.

II.4.2.4 Peran LLM dalam Konstruksi KG Modern

Perkembangan terbaru menunjukkan adanya perubahan pendekatan dalam konstruksi KG. Metode lama seperti BiLSTM-CRF dan LSTM-CNN-CRF cukup baik untuk

ekstraksi entitas dan relasi, tetapi masih membutuhkan anotasi data manual yang besar (Zhou2026).

Sebaliknya, pendekatan berbasis LLM mampu mengurangi kebutuhan anotasi karena proses ekstraksi dapat dilakukan melalui *prompt*. Oarga2026<empty citation> menyebut bahwa LLM saat ini menjadi pendekatan *state-of-the-art* untuk tugas NER dan ekstraksi KG tanpa supervisi.

Namun demikian, penggunaan LLM masih menghadapi tantangan berupa variasi hasil keluaran dan sensitivitas terhadap kualitas *prompt*. Dari sisi performa, Zhou2026<empty citation> menunjukkan bahwa kualitas KG berpengaruh langsung terhadap kinerja sistem. Kombinasi LLM dan KG yang telah dioptimasi mencapai akurasi 90,57%, lebih tinggi dibandingkan RAG berbasis dokumen (86,95%) maupun LLM tanpa pengetahuan eksternal (55,79%).

Pada penelitian tersebut, KG yang dibangun berhasil mengekstraksi 1.618 entitas dan 753 relasi dari 15 dokumen teknis dengan *entity mention coverage* sebesar 0,948 dan *relation evidence matching rate* sebesar 0,766.

II.4.3 Keunggulan Representasi Berbasis Graf dan Integrasi dengan LLM

Menurut Pan2024<empty citation>, integrasi antara LLM dan KG membentuk pendekatan baru yang menggabungkan kemampuan penalaran bahasa alami dari LLM dengan pengetahuan terstruktur dari KG. Kombinasi ini membantu mengurangi halusinasi karena model memperoleh informasi yang dapat ditelusuri dan diverifikasi.

1. Mendukung penalaran multi-hop KG memungkinkan sistem mengikuti hubungan antar entitas secara bertahap. Sebagai contoh:

`DataFrame.apply()` → `row_by_row_processing` → `high_memory_usage` → `MemoryError`

Rantai hubungan tersebut membantu sistem memahami penyebab masalah secara lebih lengkap. Mekanisme seperti ini sulit dicapai hanya menggunakan pencarian berbasis kemiripan vektor.

2. Mampu merepresentasikan struktur hierarki domain Sebagai contoh:

`pandas.DataFrame` → `pandas.NDFrame` → `numpy.ndarray`

Hubungan hierarkis tersebut penting untuk memahami keterkaitan antarobjek dalam domain pemrograman.

3. Mendukung pemberian bobot kualitas informasi

Berbeda dengan representasi vektor biasa, KG memungkinkan setiap simpul dan relasi memiliki bobot tertentu yang menunjukkan tingkat kepercayaan atau kepentingan informasi.

Tao2026<empty citation> menunjukkan bahwa penambahan KG pada sistem RAG dapat meningkatkan performa secara bertahap, mulai dari RAG konvensional menuju KG-RAG hingga pendekatan yang menggabungkan struktur relasional dan preferensi semantik. Temuan tersebut menunjukkan bahwa KG menjadi komponen penting untuk meningkatkan keandalan sistem berbasis LLM, terutama pada domain teknis seperti pemrograman.

II.5 Graph-based Retrieval-Augmented Generation

II.5.1 Motivasi dan Definisi

Graph-based Retrieval-Augmented Generation (GraphRAG) berkembang sebagai respons terhadap keterbatasan *Vector RAG* dalam menangani kueri yang memerlukan penalaran relasional dan hubungan antar entitas. Pada RAG konvensional, proses pengambilan informasi umumnya bergantung pada kemiripan vektor sehingga lebih efektif untuk menemukan dokumen yang relevan secara topikal, tetapi masih terbatas dalam menangkap hubungan semantik yang kompleks.

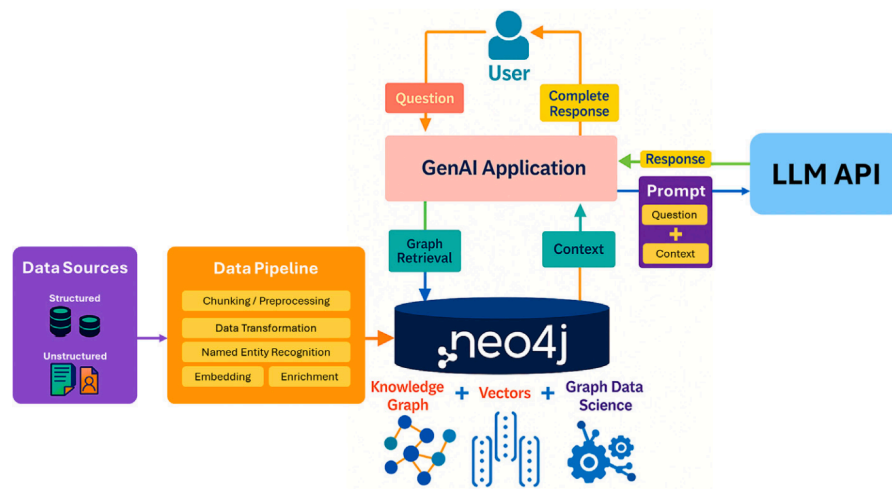
Kamalipour2026<empty citation> mendefinisikan GraphRAG sebagai paradigma yang mengintegrasikan proses *retrieve-then-generate* dengan representasi pengetahuan berbasis graf. Dalam pendekatan ini, simpul (*nodes*) merepresentasikan entitas atau konsep, sedangkan sisi (*edges*) menggambarkan relasi di antara entitas tersebut. Struktur ini memungkinkan proses pengambilan informasi yang lebih presisi, mudah dijelaskan (*explainable*), dan kaya konteks, khususnya untuk pertanyaan yang memerlukan penalaran multi-hop.

Pan2024<empty citation> menjelaskan bahwa integrasi antara LLM dan *knowledge graph* membentuk paradigma baru yang meningkatkan kemampuan penalaran berbasis pengetahuan sekaligus membantu mengurangi halusinasi faktual. Sejalan dengan itu, **Zhu2026<empty citation>** mengidentifikasi tiga komponen utama GraphRAG, yaitu: (1) representasi pengetahuan berbasis graf, (2) mekanisme *retrieval* yang mendukung penalaran multi-hop, dan (3) integrasi pengetahuan yang mempertahankan struktur relasional.

II.5.2 Arsitektur Pipeline GraphRAG

Kamalipour2026<empty citation> mendeskripsikan GraphRAG sebagai serangkaian proses yang dimulai dari pembangunan *knowledge graph*, pengambilan in-

formasi berbasis graf, hingga generasi jawaban menggunakan LLM. Alur umum tersebut ditunjukkan pada Gambar ??.



Gambar II.6 Gambaran umum *pipeline* Graph-RAG: ekstraksi triple, penelusuran subgraf (*sub-graph traversal*), serta integrasi hibrida antara representasi vektor dan simbolik.(Kamalipour2026)

II.5.2.1 Konstruksi *Knowledge Graph*.

Tahap awal dilakukan dengan membagi korpus sumber menjadi beberapa segmen yang lebih kecil dan mudah diproses. Selanjutnya, LLM atau pendekatan *Open Information Extraction* (OpenIE) digunakan untuk mengekstraksi tripel *subject-predicate-object* (SPO). Tripel-tripel tersebut kemudian ditransformasikan menjadi simpul dan relasi dalam graf, lalu disimpan pada *graph database* seperti Neo4j. Selain itu, representasi vektor (*embedding*) untuk setiap simpul dapat ditambahkan untuk mendukung mekanisme pengambilan hibrid antara graf dan vektor (Kamalipour2026).

II.5.2.2 *Retrieval* dan Penalaran Berbasis Graf.

Pada tahap ini, sistem memetakan entitas yang muncul pada kueri pengguna ke simpul yang sesuai melalui proses *Named Entity Recognition* (NER) dan *entity linking*. Setelah entitas awal diperoleh, proses pengambilan informasi dilakukan menggunakan tiga mekanisme utama (Kamalipour2026).

Pertama, *Multi-hop Traversal*, yaitu penelusuran rantai relasi antar entitas untuk membangun konteks yang lebih lengkap. Kedua, *Subgraph Extraction*, yaitu pengambilan subgraf lokal di sekitar entitas yang relevan. Ketiga, *Community Detection*, yaitu identifikasi kelompok entitas yang saling terhubung untuk mendukung pertanyaan global atau proses ringkasan. Pendekatan ini memungkinkan sistem menangkap hubungan semantik yang tidak dapat direpresentasikan hanya melalui ke-

miripan vektor.

II.5.2.3 Integrasi Konteks Berbasis Struktur.

Subgraf hasil proses *retrieval* selanjutnya dikonversi menjadi representasi tekstual yang tetap mempertahankan hubungan antar entitas. Representasi tersebut disusun menjadi *structured prompt* sebelum dikirimkan ke LLM.

Dengan mempertahankan struktur relasional, LLM memperoleh konteks yang lebih kaya dibandingkan ketika hanya menerima potongan teks tidak terstruktur. Akibatnya, jawaban yang dihasilkan menjadi lebih konsisten, dapat ditelusuri, dan berpotensi mengurangi halusinasi.

II.5.2.4 Perbandingan Kuantitatif: Vector RAG dan GraphRAG

Perbedaan utama antara Vector RAG dan GraphRAG terletak pada mekanisme pengambilan informasi. Vector RAG mengandalkan pencarian berbasis kemiripan vektor, sedangkan GraphRAG memanfaatkan struktur relasional melalui traversal graf. Ringkasan perbandingan keduanya disajikan pada Tabel ??.

Kamalipour2026<empty citation> menunjukkan bahwa GraphRAG memberikan keunggulan yang lebih nyata pada pertanyaan kompleks yang memerlukan penalaran multi-hop. Pada beberapa eksperimen di domain keuangan, kesehatan, dan hukum, penambahan *knowledge graph* meningkatkan proporsi jawaban benar sekitar 30% dibandingkan pendekatan berbasis pengambilan vektor saja.

Meskipun demikian, GraphRAG juga memiliki keterbatasan. Kualitas hasil sangat bergantung pada akurasi konstruksi graf dan keberhasilan proses *entity linking*. Kesalahan pada tahap ini dapat menyebabkan hilangnya jalur penalaran sehingga informasi penting tidak ikut diambil.

II.5.3 Bukti Empiris Efektivitas GraphRAG

Berbagai penelitian menunjukkan bahwa integrasi struktur graf dapat meningkatkan akurasi sekaligus menurunkan tingkat halusinasi pada sistem berbasis LLM.

II.5.3.1 Pythia-RAG (Ali2026).

Pythia-RAG membangun *Multimodal Knowledge Graph* (MMKG) dari sumber tekstual dan visual, kemudian mengambil subgraf yang relevan menggunakan algoritma *Prize-Collecting Steiner Tree*. Subgraf tersebut diproses lebih lanjut menggunakan Graphormer untuk mempertahankan struktur relasional.

Evaluasi pada dataset ScienceQA dan MultiModalQA menunjukkan peningkatan

Tabel II.1 Perbandingan *trade-off* antara Vector RAG dan GraphRAG berdasarkan dimensi teknis yang relevan untuk pengembangan *framework*. Bersumber dari **Kamalipour2026**<empty citation>, Tabel 8.

Dimensi	Vector RAG	GraphRAG
<i>Memory footprint</i>	Gigabyte vektor padat; perlu <i>re-embedding</i> seluruh korpus saat ada perubahan	Lebih ringan: simpul + sisi (opsional <i>node embedding</i>); pembaruan cukup dengan menyisipkan tripel baru
Inti <i>retrieval</i>	ANN berbasis kemiripan (HNSW, IVF-PQ, <i>k</i> -NN)	Traversal graf multi-hop (BFS, <i>Greedy-Prune</i> , SPARQL)
<i>Recall</i> vs presisi	<i>Recall</i> tinggi, presisi lebih rendah; memerlukan <i>reranking</i> atau kompresi tambahan	Presisi lebih tinggi pada kueri relasional; <i>recall</i> turun jika <i>entity linking</i> gagal
<i>Explainability</i>	Sulit: hanya skor kemiripan yang tersedia	Jalur penuh: entitas → relasi → entitas; dapat diaudit untuk domain teregulasi
Profil latensi	Datar terhadap panjang jalur; <100 ms dengan HNSW	Meningkat seiring jumlah <i>hop</i> ; umumnya $\sim 3 \times$ Vector untuk 3- <i>hop</i>
<i>Benchmark</i> unggulan	BEIR, NaturalQuestions (<i>single-hop</i> QA)	HotpotQA, MultiHop-RAG, RobustQA (kueri kompleks multi- <i>hop</i>)
Kegagalan tipikal	Potongan relevan secara topik tetapi tidak relevan secara konteks → halusinasi	Simpul antara yang hilang → kesenjangan jawaban
Tepat digunakan saat	Throughput tinggi, pencarian <i>single-hop</i> , latensi kritis	Penalaran multi- <i>hop</i> , auditabilitas, pembaruan terstruktur berkala

akurasi relatif masing-masing sebesar 5,4% dan 4,8%. Selain itu, *hallucination rate* berhasil ditekan menjadi 8,3%, lebih rendah dibandingkan GPT-4o (17,6%) dan MuRAG (14,2%). Hasil ini menunjukkan bahwa penggunaan KG sebagai sumber *grounding* mampu mengurangi halusinasi secara signifikan.

II.5.3.2 PRIME (Wang2026).

PRIME (*Progressive Reasoning via Incremental Multi-hop Exploration*) mengusulkan mekanisme penalaran bertahap berbasis subgraf yang memungkinkan eksplorasi hubungan semantik secara progresif. Metode ini mencapai performa sebesar 91,6% Hit@1 pada WebQSP dan 78,3% F1 pada CWQ, sekaligus menunjukkan efisiensi komputasi yang tinggi karena tidak memerlukan pelatihan tambahan.

II.5.3.3 LARK (BenKhalifa2026).

LARK (*License Analysis with RAG and Knowledge Graphs*) menerapkan GraphRAG pada analisis lisensi perangkat lunak. Evaluasi terhadap sekitar 4.000 proyek Python

sumber terbuka menunjukkan akurasi deteksi lisensi sebesar 98,1% dan F1 kompatibilitas sebesar 96,2%. Tingkat halusinasi yang diperoleh hanya 2,1%, karena KG diposisikan sebagai lapisan pengetahuan yang bersifat otoritatif dan dapat diaudit, sementara LLM dibatasi pada proses pemahaman kueri dan ekstraksi informasi.

II.5.4 Aplikasi GraphRAG pada Domain Teknis

Berbagai studi menunjukkan bahwa GraphRAG memberikan manfaat terbesar pada domain yang memiliki struktur pengetahuan kuat dan memerlukan penalaran relasional.

Awotunde²⁰²⁶^{<empty citation>} menerapkan pendekatan KG + RAG pada *text-to-SQL generation* untuk basis data spesifik-domain dan menemukan bahwa pendekatan tersebut mampu mengatasi keterbatasan RAG konvensional maupun *fine-tuning*. Sementara itu, Cong²⁰²⁶^{<empty citation>} menunjukkan bahwa struktur relasional dalam KG meningkatkan kualitas pengambilan konteks pada proses ideasi produk.

Karakteristik tersebut sangat mirip dengan domain pemrograman yang menjadi fokus penelitian ini. Pengetahuan pemrograman bersifat terstruktur, melibatkan entitas seperti fungsi, kelas, pustaka, serta relasi seperti *requires*, *raises*, dan *solves*. Selain itu, banyak pertanyaan teknis memerlukan penalaran multi-hop untuk menemukan akar masalah dan solusi yang tepat.

Stack Overflow menyediakan sumber pengetahuan komunitas yang kaya sekaligus memiliki sinyal kualitas berupa skor, jawaban terpilih, dan interaksi pengguna. Informasi ini dapat dimanfaatkan sebagai bobot kepercayaan dalam *knowledge graph*, sehingga mendukung proses pengambilan informasi yang lebih akurat.

II.5.5 Sintesis dan Posisi Penelitian

Berdasarkan kajian literatur, Vector RAG memiliki keunggulan pada skalabilitas dan kemampuan pengambilan informasi berbasis topik. Namun, pendekatan ini masih terbatas dalam menangani hubungan relasional dan penalaran multi-hop.

Sebaliknya, GraphRAG menyediakan struktur pengetahuan yang dapat dijelaskan, mendukung penalaran bertingkat, dan menghasilkan konteks yang lebih kaya. Kelemahannya terletak pada kebutuhan konstruksi KG dan sensitivitas terhadap kesalahan *entity linking*.

Penelitian ini memposisikan diri sebagai pengembangan lebih lanjut dari GraphRAG dengan menambahkan sinyal kepercayaan komunitas Stack Overflow sebagai bobot pada *knowledge graph*. Pendekatan tersebut diharapkan mampu meningkatkan kua-

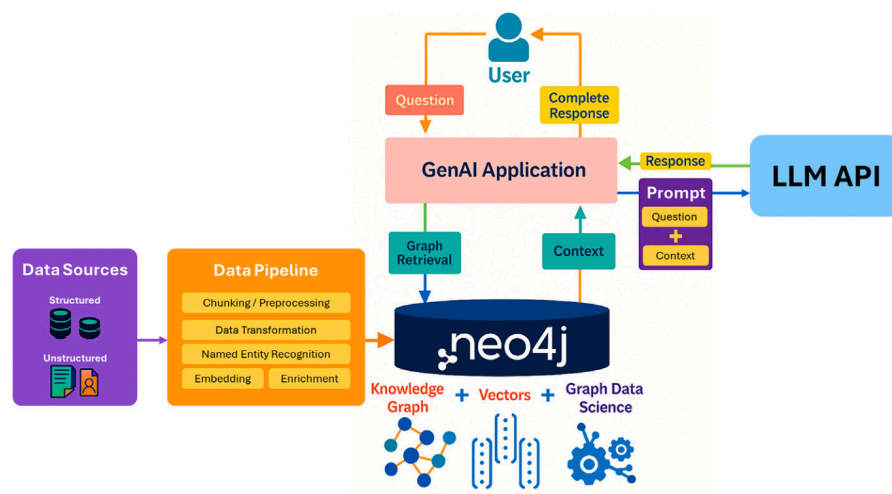
litas *retrieval*, memperkuat proses *grounding*, dan mengurangi halusinasi pada LLM untuk domain pemrograman.

II.6 Stack Overflow sebagai Basis Pengetahuan Tervalidasi Sosial

II.6.1 Struktur dan Mekanisme Kurasi Komunitas

Stack Overflow adalah platform *question and answer* (Q&A) yang berfokus pada pemrograman dan pengembangan perangkat lunak. Per 2024, platform ini menyimpan lebih dari 58 juta pasang pertanyaan dan jawaban (DaSilva2025). Yang membedakan Stack Overflow dari repositori teks biasa adalah mekanisme kurasi sosial bawaan yang menghasilkan sinyal kualitas terukur dengan sistem *upvote/downvote*, status *accepted answer*, dan sistem reputasi kontributor.

DaSilva2025<empty citation> merancang studi empiris yang secara eksplisit memanfaatkan tiga mekanisme ini sebagai proksi kualitas jawaban (Gambar ??). Studi tersebut mengonfirmasi bahwa jawaban dengan skor tinggi dan status *accepted* memiliki korelasi positif dengan akurasi teknis yang terverifikasi. Hal ini menjadikan Stack Overflow sebagai basis pengetahuan tervalidasi secara sosial (*socially-validated knowledge base*).



Gambar II.7 Gambaran umum rancangan empiris yang digunakan untuk menguji korelasi antara sinyal kepercayaan komunitas Stack Overflow dengan akurasi jawaban LLM. (DaSilva2025)

II.6.2 Sinyal Kepercayaan sebagai Proksi Kualitas

Dalam konteks *knowledge graph* berbobot, sinyal kepercayaan yang tersedia pada Stack Overflow dapat dimanfaatkan sebagai bobot pada simpul dan relasi graf. Dengan demikian, kualitas informasi tidak hanya ditentukan oleh keberadaan entitas dan hubungan, tetapi juga oleh tingkat kepercayaan komunitas terhadap informasi

tersebut. Tabel ?? menunjukkan sinyal kepercayaan yang digunakan beserta interpretasinya pada KG berbobot.

Tabel II.2 Sinyal kepercayaan komunitas Stack Overflow dan interpretasinya sebagai bobot pada *knowledge graph* berbobot

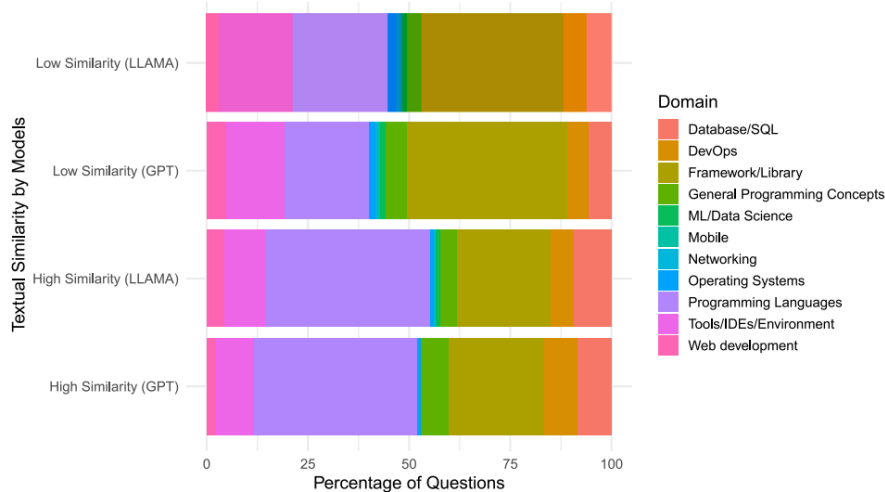
Sinyal	Sumber	Interpretasi sebagai Bobot
Skor jawaban	Akumulasi <i>upvote</i> – <i>downvote</i>	Bobot simpul jawaban: penerimaan kolektif komunitas
Status <i>accepted answer</i>	Pilihan eksplisit penanya	Bobot biner: validasi langsung oleh praktisi
Reputasi kontributor	Rekam jejak kontribusi historis	Bobot kepercayaan pada sisi kontribusi
Skor pertanyaan	Akumulasi <i>upvote</i> pada pertanyaan	Bobot simpul pertanyaan: relevansi masalah
Jumlah tampilan (<i>view count</i>)	Aktivitas baca pengguna	Bobot popularitas: relevansi jangka panjang

II.6.3 Pemanfaatan Konten Stack Overflow dalam Sistem AI

Beberapa penelitian telah memanfaatkan Stack Overflow sebagai sumber pengetahuan untuk meningkatkan kualitas keluaran model AI. **Nashid2023**<empty citation> menunjukkan bahwa penggunaan konten Stack Overflow sebagai sumber konteks dapat meningkatkan kualitas jawaban LLM pada tugas pemrograman. Namun, pendekatan tersebut masih menggunakan RAG konvensional berbasis kemiripan vektor sehingga belum memanfaatkan struktur relasional maupun sinyal kepercayaan komunitas.

Selain itu, **DaSilva2025**<empty citation> menunjukkan bahwa tingkat kesesuaian antara jawaban LLM dan jawaban komunitas tidak sama pada semua domain pemrograman (Gambar ??). Beberapa domain menghasilkan kemiripan tinggi, sementara domain lain menunjukkan perbedaan yang cukup besar. Temuan ini mengindikasikan bahwa halusinasi LLM bersifat tidak merata antar domain dan perlu dipertimbangkan pada proses evaluasi.

Temuan tersebut diperkuat oleh **Kabir2024**<empty citation>, yang melakukan studi terhadap jawaban ChatGPT pada pertanyaan Stack Overflow. Penelitian ini menunjukkan bahwa keluaran LLM masih mengandung ketidakakuratan faktual yang cukup signifikan sehingga aspek keandalan perlu menjadi perhatian utama pada penerapan sistem berbasis LLM.



Gambar II.8 Distribusi domain pertanyaan untuk jawaban LLM dengan kemiripan tinggi dan rendah terhadap komunitas Stack Overflow. DaSilva2025<empty citation>

II.7 Evaluasi Keandalan *Large Language Model*

II.7.1 Tantangan Pengukuran Halusinasi

Evaluasi keandalan LLM bukan merupakan tugas yang sederhana. Gu2026<empty citation> menjelaskan bahwa proses pengukuran memerlukan desain dan standarisasi evaluasi yang baik. Evaluasi manual berbasis anotasi manusia umumnya menghasilkan penilaian yang akurat, tetapi sulit diterapkan pada skala besar karena membutuhkan waktu dan biaya yang tinggi. Sebaliknya, evaluasi berbasis *benchmark* lebih mudah diskalakan, namun berpotensi mengalami kontaminasi data apabila dataset evaluasi pernah digunakan pada proses pelatihan model.

II.7.2 Metrik Evaluasi Halusinasi

Beberapa metrik kuantitatif yang relevan untuk penelitian ini adalah sebagai berikut.

II.7.2.1 Akurasi Faktual (*Factual Accuracy*).

Proporsi klaim faktual dalam jawaban yang dapat diverifikasi sebagai benar terhadap sumber referensi. Dalam domain pemrograman, sumber referensi dapat berupa dokumentasi resmi, kode yang dapat dieksekusi, atau jawaban komunitas SO berskor tinggi.

II.7.2.2 Tingkat Halusinasi (*Hallucination Rate*).

Proporsi jawaban yang mengandung setidaknya satu klaim halusinasi, diukur pada dua level: per-jawaban dan per-klaim (Ji2023; Huang2025).

II.7.2.3 Skor Konsistensi (*Consistency Score*).

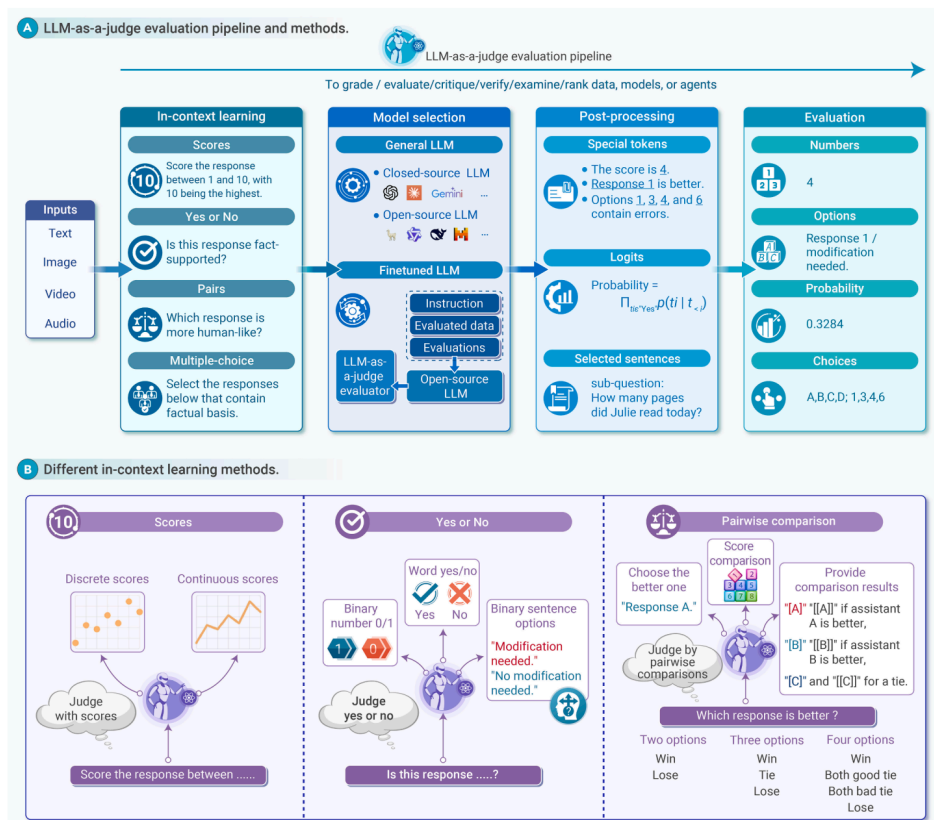
Stabilitas jawaban model terhadap pertanyaan yang semantis serupa namun diformulasikan berbeda (Chang2026). Relevan karena Jiang2026<empty citation> mendokumentasikan bahwa *prompt sensitivity*—ketidakstabilan respons LLM terhadap perubahan kecil formulasi—adalah masalah utama dalam aplikasi pemrograman.

II.7.3 Paradigma *LLM-as-a-Judge*

Salah satu pendekatan yang banyak digunakan untuk mengevaluasi keluaran LLM adalah paradigma *LLM-as-a-Judge*.

Menurut Gu2026<empty citation>, pendekatan ini menggunakan LLM yang lebih kuat sebagai penilai otomatis terhadap keluaran model lain. Kerangka evaluasi tersebut terdiri dari empat tahap diilustrasikan pada Gambar ??, yaitu:

1. pemberian konteks dan instruksi evaluasi;
2. proses inferensi model penilai;
3. *post-processing* hasil evaluasi;
4. agregasi skor untuk meningkatkan reliabilitas.



Gambar II.9 (A) *Pipeline* evaluasi *LLM-as-a-judge* dari awal hingga akhir, mencakup empat tahap. (B) Metode penilaian: *scoring*, *classification*, dan *ranking*. Gu2026<empty citation>

Untuk meningkatkan keandalan penilaian, **Gu2026** mengusulkan beberapa strategi seperti penggunaan rubrik evaluasi terstruktur, pendekatan *Chain-of-Thought*, dan kombinasi beberapa model penilai (*ensemble*).

Pendekatan ini relevan digunakan pada penelitian ini karena evaluasi manual terhadap akurasi faktual dalam jumlah besar tidak praktis dilakukan secara langsung.

II.8 Sintesis Penelitian Terkait dan *Research Gap*

II.8.1 Analisis Kritis Penelitian Terkait

Tinjauan terhadap penelitian terdahulu mengungkapkan empat kluster aktif yang masing-masing hanya menjawab sebagian dari permasalahan yang diidentifikasi. Tabel ?? merangkum posisi masing-masing kluster terhadap tiga dimensi yang menjadi fokus penelitian ini: penggunaan sumber pengetahuan komunitas StackOverflow, pemanfaatan sinyal kepercayaan komunitas sebagai bobot, dan penerapan GraphRAG pada domain pemrograman.

Tabel II.3 Posisi penelitian terkait terhadap tiga dimensi utama penelitian ini

Penelitian	Fokus	SO sebagai sumber	Sinyal kepercayaan	GraphRAG
DaSilva2025	Analisis komparatif LLM vs SO	✓ (benchmark)	×	×
Kabir2024	Karakteristik jawaban ChatGPT	✓ (benchmark)	×	×
Alomari2026	LLM untuk kualitas kode	×	×	×
Jiang2026	Deteksi duplikasi kode	×	×	×
Nashid2023	RAG berbasis konten SO	✓ (konteks)	×	×
Wang2026	GraphRAG (PRIME)	×	×	✓
Ali2026	GraphRAG multi-modal	×	×	✓
Zhou2026	LLM domain-spesifik + RAG	×	×	✓
Penelitian ini	GraphRAG berbasis SO	✓✓	✓✓	✓✓

II.8.1.1 Kluster 1: Evaluasi Komparatif LLM vs Komunitas

DaSilva2025 dan **Kabir2024** menetapkan *baseline* kuantitatif tentang keterbatasan LLM dibandingkan keahlian komunitas StackOverflow. **DaSilva2025** mengonfirmasi bahwa ChatGPT-3.5 dan LLaMA-2 belum melampaui keahlian komunitas pada domain *Frameworks and Libraries*, omain yang paling banyak dikunjungi pengembang dan mendokumentasik-

an penurunan aktivitas *posting* yang signifikan dan berkelanjutan di StackOverflow pasca peluncuran ChatGPT. **Kabir2024**<empty citation> mengidentifikasi bahwa lebih dari 50% jawaban ChatGPT pada pertanyaan StackOverflow mengandung ketidakakuratan yang cukup serius untuk memengaruhi pengambilan keputusan teknis.

Nilai kedua studi ini terletak pada penetapan *ground truth* empiris yang kuat. Namun, orientasinya yang sepenuhnya evaluatif menempatkannya sebagai diagnosis masalah tanpa resep solusi arsitektural: tidak ada mekanisme yang diusulkan untuk mengintegrasikan pengetahuan komunitas SO ke dalam LLM secara terstruktur.

II.8.1.2 Kluster 2: LLM untuk Kualitas Kode

Alomari2026<empty citation> mengkonfirmasi melalui tinjauan literatur sistematis bahwa LLM menghasilkan tingkat halusinasi 76,3% pada tugas refaktorisasi kode, dan **Jiang2026**<empty citation> menunjukkan bahwa *prompt sensitivity* menyebabkan variasi jawaban yang substansial pada deteksi duplikasi kode. Kedua studi berfokus pada teknik *prompting* dan *fine-tuning* sebagai strategi mitigasi. **Ronanki2026**<empty citation> dari sisi industri mengidentifikasi *extrinsic hallucination* sebagai hambatan utama adopsi LLM dalam SE industrial, namun menyarankan RAG hanya secara umum tanpa mengusulkan arsitektur spesifik berbasis KG.

Tidak ada penelitian dalam kluster ini yang membahas GraphRAG sebagai strategi *grounding*, dan tidak ada yang memanfaatkan pengetahuan komunitas tervalidasi sebagai fondasi mitigasi halusinasi.

II.8.1.3 Kluster 3: RAG Berbasis Konten Stack Overflow

Nashid2023<empty citation> membuktikan nilai augmentatif konten Stack Overflow dengan menunjukkan bahwa pengambilan konteks dari StackOverflow secara signifikan meningkatkan kualitas jawaban LLM pada tugas pemrograman. Ini merupakan bukti langsung bahwa repositori pengetahuan komunitas StackOverflow memiliki nilai augmentatif yang terukur bagi LLM.

Namun, pendekatan ini menggunakan *dense retrieval* berbasis kemiripan vektor murni. **Zhou2026**<empty citation> telah menunjukkan bahwa pendekatan vektor saja menghasilkan akurasi hanya 86,95%—dibandingkan 90,57% ketika dikombinasikan dengan KG terstruktur. Lebih jauh, **Nashid2023**<empty citation> tidak memanfaatkan sinyal kepercayaan komunitas (*upvote*, *accepted answer*, reputasi kontributor) sebagai bobot dalam proses pengambilan konteks, sehingga seluruh potongan teks StackOverflow diperlakukan setara tanpa mempertimbangkan kualitas yang sudah divalidasi komunitas.

II.8.1.4 Kluster 4: GraphRAG pada Domain Teknis

Wang2026<empty citation> (PRIME), **Ali2026<empty citation>** (PythiaRAG), dan **Zhou2026<empty citation>** (IC-HRAG) membuktikan efektivitas GraphRAG pada berbagai domain teknis. PRIME mencapai 91,6% Hit@1 pada WebQSP dengan latensi $1.000\times$ lebih cepat dari metode berbasis pelatihan. PythiaRAG mereduksi *hallucination rate* dari 17,6% (GPT-4o) menjadi 8,3%. IC-HRAG mencapai presisi 97,22% pada domain peralatan industri.

Namun, seluruh penelitian dalam kluster ini menggunakan *knowledge graph* yang dibangun dari korpus dokumen teknis generik atau teks tidak terstruktur domain-spesifik—bukan dari komunitas *question answering* dengan sinyal validasi sosial yang terstruktur. **Chang2026<empty citation>** menegaskan bahwa GraphRAG masih menghadapi tantangan utama dalam mempertahankan informasi struktural dan menangani bukti *multi-hop* yang tidak lengkap atau berisik—dua tantangan yang dapat diatasi secara unik oleh sinyal kepercayaan komunitas StackOverflow sebagai mekanisme penyaringan kualitas bawaan.

Zhou2026<empty citation> sendiri mengakui bahwa penerapan GraphRAG pada domain-domain lain masih terbuka sebagai arah penelitian lanjutan, dan **Chang2026<empty citation>** secara eksplisit mengidentifikasi integrasi GraphRAG dengan sumber pengetahuan komunitas tervalidasi sebagai jalur teknis yang menjanjikan untuk meningkatkan *grounding* faktual LLM.

II.8.2 Identifikasi Kesenjangan

Sintesis dari empat kluster literatur menunjukkan adanya tiga kesenjangan metodologis utama yang menjadi dasar pengembangan penelitian ini.

1. Tidak adanya mekanisme *grounding* arsitektural berbasis komunitas Stack Overflow

Studi evaluatif oleh **DaSilva2025<empty citation>** dan **Kabir2024<empty citation>** telah menunjukkan keterbatasan LLM dibandingkan komunitas Stack Overflow dalam menjawab pertanyaan pemrograman. Namun, penelitian tersebut hanya berfokus pada evaluasi performa dan belum mengusulkan solusi pada tingkat arsitektur sistem.

Dengan demikian, pertanyaan mengenai bagaimana merancang sistem yang mampu mengintegrasikan pengetahuan komunitas Stack Overflow ke dalam mekanisme kerja LLM masih belum terjawab. Kondisi ini menunjukkan adanya kesenjangan antara hasil evaluasi empiris dan implementasi arsitektur solusi.

2. RAG berbasis Stack Overflow belum memanfaatkan struktur relasional dan sinyal kepercayaan komunitas

Nashid2023<empty citation> menunjukkan bahwa konten Stack Overflow dapat meningkatkan kualitas jawaban LLM ketika digunakan sebagai sumber konteks. Namun, pendekatan yang digunakan masih berupa RAG konvensional berbasis *dense retrieval*, sehingga hanya mempertimbangkan kemiripan vektor.

Pendekatan tersebut belum memanfaatkan hubungan relasional antar entitas maupun sinyal kepercayaan komunitas seperti *upvote*, *accepted answer*, dan reputasi kontributor. Padahal, sinyal-sinyal tersebut merupakan karakteristik unik Stack Overflow yang tidak dimiliki oleh repositori teks biasa. Studi **Zhou2026**<empty citation> juga menunjukkan bahwa keterbatasan representasi relasional menyebabkan akurasi RAG konvensional berhenti pada tingkat 86,95%.

3. GraphRAG belum diterapkan pada *knowledge graph* berbasis komunitas teknis tervalidasi sosial di domain pemrograman

Penelitian oleh **Wang2026**<empty citation>, **Ali2026**<empty citation>, dan **Zhou2026**<empty citation> membuktikan bahwa GraphRAG mampu meningkatkan kemampuan penalaran dan akurasi pada berbagai domain teknis. Namun, sumber pengetahuan yang digunakan masih berasal dari dokumen teknis atau korpus umum tanpa memasukkan sinyal kualitas sosial. Berdasarkan tinjauan literatur, belum ditemukan penelitian yang secara khusus:

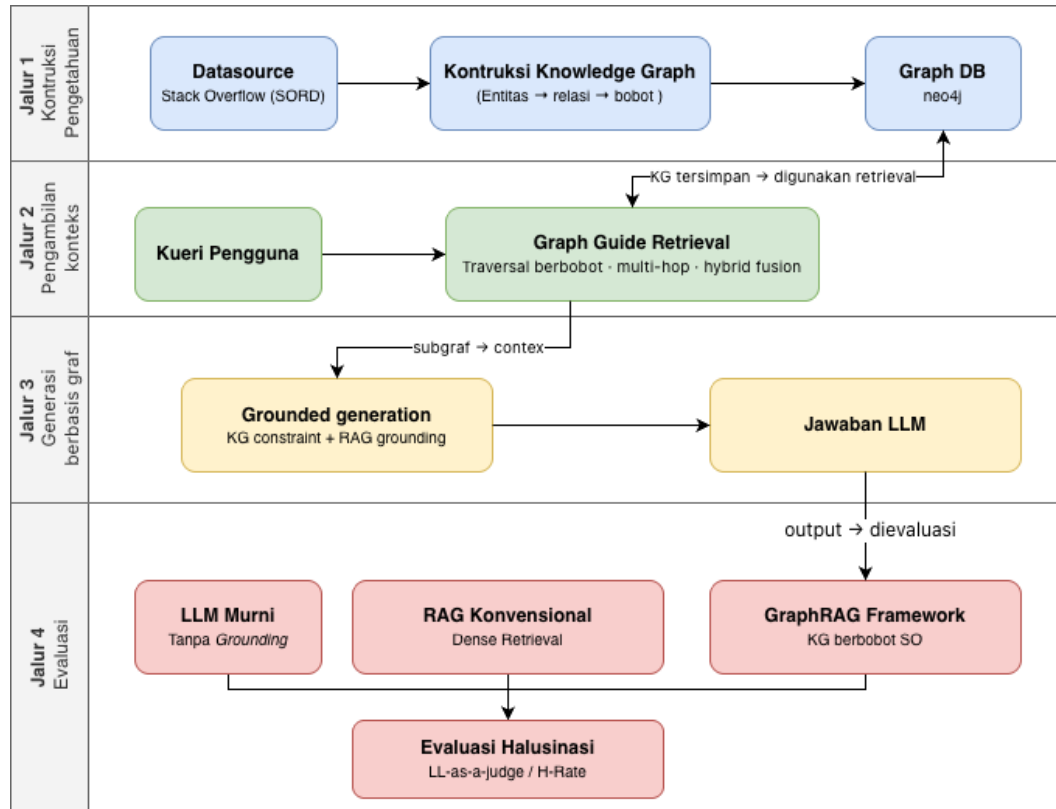
- (a) membangun *knowledge graph* dari konten Stack Overflow;
- (b) merepresentasikan sinyal kepercayaan komunitas sebagai bobot eksplisit pada simpul dan relasi graf;
- (c) menggunakan graf berbobot tersebut sebagai sumber pengetahuan utama dalam GraphRAG untuk domain pemrograman.

Chang2026<empty citation> menyebutkan bahwa arah penelitian ini memiliki potensi yang tinggi, namun masih belum banyak dieksplorasi dalam literatur yang tersedia.

Ketiga kesenjangan tersebut menjadi dasar pengembangan penelitian ini, yaitu membangun *GraphRAG Framework* yang mengintegrasikan *knowledge graph* berbobot dari komunitas Stack Overflow ke dalam *pipeline* generasi jawaban LLM pada domain pemrograman. Pendekatan ini diharapkan dapat menggabungkan keunggulan struktur relasional GraphRAG dengan validasi sosial yang dimiliki oleh komunitas Stack Overflow.

II.8.3 Kerangka Konseptual Penelitian

Berdasarkan studi literatur, kerangka konseptual penelitian ini (Gambar ??) mengorganisasi komponen-komponen teoritis ke dalam empat jalur yang saling terhubung secara sekuensial, yaitu (1) konstruksi pengetahuan, (2) pengambilan konteks, (3) generasi berbasis graf, dan (4) evaluasi.



Gambar II.10 Kerangka konseptual penelitian

Jalur 1 - Konstruksi pengetahuan. Jalur pertama membangun fondasi pengetahuan terstruktur melalui tiga aktivitas berurutan. Pertama, data diekstraksi dari *Stack Exchange Data Dump* (SORD) yang mencakup pertanyaan, jawaban, tag, dan metadata komunitas (Xu2024). Kedua, data tersebut diproses melalui pipeline konstruksi Knowledge Graph (KG) berbasis LLM yang meliputi proses Named Entity Recognition (NER) dan ekstraksi triple (subjek–predikat–objek). Pada tahap ini, bobot komunitas seperti upvote, accepted answer, dan reputasi kontributor diintegrasikan sebagai atribut eksplisit dalam graf (Zhou2026; Cong2026). Ketiga, seluruh triple yang dihasilkan disimpan ke dalam graph database Neo4j disertai vector embedding pada setiap simpul untuk mendukung mekanisme pencarian hibrida (Kamalipour2026). KG yang tersimpan kemudian berfungsi sebagai indeks pengetahuan terstruktur yang digunakan dalam seluruh proses pengambilan konteks pada jalur berikutnya.

Jalur 2 - Pengambilan konteks. Jalur kedua dimulai dari kueri pengguna yang diproses melalui *Named Entity Recognition* (NER) dan *entity linking* untuk memetakan entitas pada pertanyaan ke simpul yang relevan di dalam KG. Selanjutnya, mekanisme *graph-guided retrieval* mengeksekusi traversal berbobot secara *multi-hop* pada KG yang telah dibangun. Proses traversal ini mengikuti jalur relasi dari entitas awal hingga menghasilkan subgraf kontekstual yang kaya secara relasional maupun semantik (Wang2026; Zhou2026). Subgraf tersebut kemudian digunakan sebagai konteks utama pada tahap generasi jawaban di jalur berikutnya.

Jalur 3 -Generasi berbasis graf. Subgraf yang dihasilkan pada Jalur 2 diproses melalui *grounded generation*. Pada tahap ini, setiap *triple* direpresentasikan dalam bentuk pasangan (Entitas-Relasi-Entitas),+, segmen teks sumber dan disertakan sebagai *structured prompt* ke LLM. Mekanisme *dual-constraint* diterapkan melalui dua lapisan kendali, yaitu kendala berbasis KG yang mengharuskan LLM menghasilkan penjelasan sesuai fakta yang direpresentasikan oleh graf, serta mekanisme *RAG grounding* yang mengaitkan setiap penjelasan dengan sumber tekstual yang dapat diverifikasi (BenKhalifa2026). Pendekatan ini memungkinkan sistem menghasilkan jawaban dengan akurasi faktual yang lebih tinggi sekaligus menyediakan jalur *provenance* yang dapat ditelusuri kembali ke simpul KG berbasis Stack Overflow (Ali2026).

Jalur 4 - Evaluasi dan validasi. Jawaban LLM yang dihasilkan pada Jalur 3 dievaluasi melalui tiga skenario perbandingan yang disusun secara bertingkat, yaitu: (1) LLM murni tanpa mekanisme *grounding*, (2) RAG konvensional berbasis *dense retrieval*, dan (3) *GraphRAG Framework* yang diusulkan dengan KG berbobot dari Stack Overflow. Ketiga skenario tersebut dievaluasi menggunakan protokol berbasis paradigma *LLM-as-a-Judge* (Gu2026). Evaluasi dilakukan menggunakan sejumlah metrik, meliputi *hallucination rate* (H-Rate), *faithfulness*, *answer relevance* melalui kerangka RAGAS, serta *agreement with human judgments* yang diukur menggunakan Cohen's κ (Kamalipour2026).

Keempat jalur tersebut membentuk suatu sistem yang terintegrasi secara kohesif. KG yang dibangun pada Jalur 1 berfungsi sebagai indeks pengetahuan bagi proses retrieval pada Jalur 2, subgraf hasil retrieval menjadi konteks bagi proses generasi pada Jalur 3, sedangkan jawaban yang dihasilkan LLM menjadi objek evaluasi pada Jalur 4. Konsistensi antara permasalahan penelitian, solusi yang diusulkan, dan artefak yang dirancang merupakan prasyarat penting dalam menjaga kualitas metodologis penelitian berbasis desain (Peffer2007).

BAB III

Analisis dan Desain

Bab ini merupakan kelanjutan langsung dari analisis *research gap* yang diidentifikasi pada Bab ???. Tiga kesenjangan yang ditemukan yaitu G1 (absennya mekanisme *grounding* arsitektural berbasis komunitas Stack Overflow), G2 (RAG berbasis SO yang tidak memanfaatkan dimensi relasional dan sinyal kepercayaan komunitas), serta G3 (GraphRAG yang belum diaplikasikan pada *knowledge graph* dari komunitas teknis bervalidasi sosial). kesenjangan tersebut diterjemahkan ke dalam keputusan analisis dan desain yang konkret. Secara garis besar, bab ini membahas hal-hal berikut:

1. Deskripsi hasil analisis kondisi saat ini dan persoalan yang ditangani, merujuk pada bukti empiris dari studi-studi yang diidentifikasi pada Bab ??,
2. Analisis kebutuhan komponen *GraphRAG Framework*, mencakup kebutuhan fungsional dan nonfungsional yang diturunkan dari rumusan masalah dan hipotesis penelitian yang ditetapkan di Bab ??, dan
3. Analisis pemilihan solusi beserta desain konseptual *GraphRAG Framework* yang merespons ketiga kesenjangan secara sistematis, termasuk rancangan *knowledge graph* berbobot, mekanisme *graph-guided retrieval*, dan protokol evaluasi komparatif.

III.1 Analisis Kondisi Saat Ini

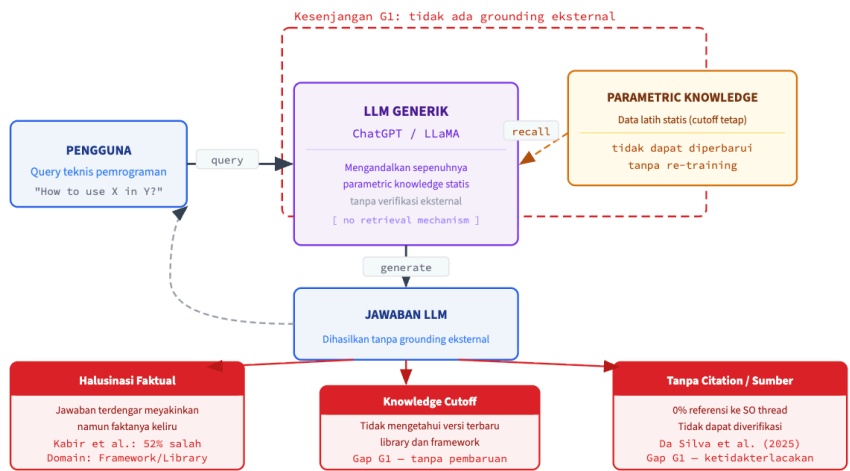
III.1.1 Model Konseptual Sistem yang Ada

Sistem tanya-jawab teknis berbasis *Large Language Model* (LLM) yang umum digunakan saat ini masih bekerja menggunakan mekanisme satu arah. Pengguna memberikan pertanyaan terkait pemrograman, kemudian model menghasilkan jawaban berdasarkan pengetahuan internal yang telah dipelajari selama proses pra-pelatihan (*pre-training*) (Huang2025). Pengetahuan tersebut dikenal sebagai *parametric knowledge*, yaitu informasi yang tersimpan dalam parameter atau bobot model. Dengan mekanisme ini, LLM menghasilkan respons berdasarkan apa yang telah dipelajari sebelumnya tanpa mengambil informasi tambahan dari sumber eksternal saat proses inferensi berlangsung.

Konsekuensinya, sistem tidak memiliki mekanisme untuk melakukan *grounding* terhadap sumber pengetahuan yang terverifikasi, seperti dokumentasi resmi, basis pe-

ngetahuan komunitas, maupun repositori teknis lainnya. Akibatnya, kualitas jawaban sangat bergantung pada cakupan dan representativitas data latih yang digunakan selama pelatihan model. Kondisi tersebut dapat menimbulkan beberapa permasalahan, seperti ketidakakuratan informasi, hilangnya konteks teknis tertentu, serta munculnya halusinasi ketika model menghadapi pertanyaan yang tidak tercakup dengan baik pada data pelatihan.

Gambar ?? memperlihatkan model konseptual sistem yang digunakan saat ini beserta titik-titik potensi kegagalan yang menjadi fokus pada penelitian ini.



Gambar III.1 Model konseptual sistem tanya-jawab berbasis LLM generik tanpa mekanisme pengambilan konteks dari sumber terverifikasi.

III.1.2 Bukti Empiris Keterbatasan Sistem Saat Ini

Studi **DaSilva2025** menyediakan bukti kuantitatif paling komprehensif tentang keterbatasan LLM ketika menangani pertanyaan komunitas Stack Overflow. Studi tersebut mengevaluasi ChatGPT-3.5 dan LLaMA-2-7b menggunakan pertanyaan dari Stack Overflow yang mencakup berbagai domain *software engineering*. Temuan yang relevan dengan penelitian ini dirangkum pada Tabel ??.

Studi **Kabir2024** memperkuat temuan ini dengan melaporkan bahwa 52% jawaban ChatGPT untuk pertanyaan SO bersifat tidak benar ketika dievaluasi secara manual, angka yang jauh lebih tinggi dari yang diklaim dalam evaluasi berbasis metrik otomatis. Ketidakakuratan ini paling parah pada pertanyaan yang membutuhkan pengetahuan tentang versi *library* atau *framework* terkini, yang merupakan manifestasi konkret dari kesenjangan G2 dan G3.

Selain itu, **Nashid2023** mendemonstrasikan bahwa augmentasi

Tabel III.1 Temuan empiris **DaSilva2025**<empty citation> yang menjadi titik acuan pengukuran *improvement* penelitian ini.

Dimensi	Temuan	Implikasi untuk Penelitian Ini
Kemiripan tekstual keseluruhan	Sekitar 85% pertanyaan memiliki <i>cosine similarity</i> > 0,5 terhadap jawaban diterima SO	15% pertanyaan—yang terkait domain Framework & Library—adalah target utama <i>improvement</i>
Domain paling lemah	<i>Frameworks and Libraries</i> : LLM gagal karena <i>knowledge cutoff</i> dan absennya <i>community conventions</i>	GraphRAG Framework harus secara eksplisit menjawab kegagalan pada domain ini
Keterlacakan sumber	LLM menghasilkan 0% referensi ke sumber SO	Setiap jawaban GraphRAG harus menyertakan kutipan ke SO thread
Dampak pada komunitas	Penurunan signifikan aktivitas posting SO pasca-rilis ChatGPT, namun pengguna kembali ke SO ketika LLM gagal	Mengkonfirmasi bahwa pengetahuan komunitas SO belum dapat digantikan LLM

konteks dari Stack Overflow meningkatkan relevansi jawaban LLM pada tugas pemrograman, namun pendekatan berbasis *dense retrieval* yang digunakan tidak mengeksplorasi struktur relasional dan sinyal kepercayaan yang tersedia dalam konten Stackoverflow. Inilah inti kesenjangan G2.

III.1.3 Identifikasi Masalah

Berdasarkan bukti empiris dari ketiga studi di atas dan sintesis di Bab ??, masalah utama yang ditangani dalam penelitian ini dijabarkan sebagai berikut:

1. Halusinasi Faktual pada Domain *Framework* dan *Library*. LLM menghasilkan jawaban yang terdengar meyakinkan namun faktanya keliru, khususnya ketika pertanyaan menyentuh *library* atau *framework* yang rilis atau diperbarui setelah batas pengetahuan (*knowledge cutoff*) model. Ini adalah manifestasi dari absennya mekanisme *grounding* eksternal (G1).
2. Kegagalan Memanfaatkan Sinyal Kepercayaan Komunitas Stackoverflow. Informasi signal kepercayaan seperti *upvote*, status *accepted answer*, reputasi

kontributor, hubungan rekomendasi yang tersedia di StackOverflow namun tidak dieksploitasi oleh LLM maupun sistem RAG konvensional. Ini adalah inti kesenjangan G2.

3. Ketidakterlacakan Sumber Jawaban. Jawaban LLM tidak dapat dilacak ke diskusi komunitas yang menjadi dasarnya. Ketidakterlacakan ini menghambat verifikasi independen dan mengurangi kepercayaan pengguna, terutama pada konteks profesional.
4. Pengetahuan yang Tidak Dapat Diperbarui Tanpa *Re-training*. LLM tidak dapat mengadaptasi pengetahuannya secara dinamis ketika ekosistem *library* berkembang, tanpa proses *fine-tuning* ulang yang mahal secara komputasi.

III.2 Analisis Kebutuhan

III.2.1 Identifikasi Kebutuhan Pengguna

Pengguna utama *GraphRAG Framework* adalah *software developer* dan *software engineer* yang menggunakan LLM untuk mendapatkan jawaban atas pertanyaan teknis di domain pemrograman, pertanyaan yang biasanya diajukan ke StackOverflow. Tabel ?? memetakan masalah pengguna ke kebutuhan solusi yang harus dipenuhi artefak.

III.2.2 Kebutuhan Fungsional

Berdasarkan pemetaan masalah di atas, kebutuhan fungsional *GraphRAG Framework* dirumuskan sebagai berikut:

- F1.** Konstruksi *Knowledge Graph* Berbobot dari SORD. Sistem harus dapat memproses dataset SORD mencakup pertanyaan, jawaban diterima, komentar, tag, dan data rekomendasi untuk mengekstrak entitas dan relasi, lalu membangun KG berbobot di mana bobot pada simpul dan sisi merepresentasikan sinyal kepercayaan komunitas SO (skor *upvote*, status *accepted answer*, reputasi kontributor, dan kekuatan rekomendasi antar-thread). Kebutuhan ini secara langsung menjawab G2 dan G3.
- F2.** Pengindeksan Vektor Teks Konten SORD Sistem harus mengindeks teks pertanyaan dan jawaban dari SORD ke dalam *vector store* menggunakan model *embedding* untuk mendukung pencarian semantik berbasis kemiripan kosinus sebagai jalur komplementer terhadap traversal graf.
- F3.** Pemahaman dan Dekomposisi Kueri. Sistem harus mampu mengekstrak entitas kunci dari kueri pengguna menggunakan *named entity recognition* (NER) dan memetakan entitas tersebut ke simpul yang ada di KG SORD.
- F4.** *Graph-Guided Retrieval* Berbasis Sinyal Kepercayaan. Mekanisme pengambilan konteks harus mempertimbangkan bobot kepercayaan saat melakukan

Tabel III.2 Pemetaan masalah pengguna ke kebutuhan solusi *GraphRAG Framework*.

No.	Masalah Pengguna	Akar Penyebab (Gap)	Kebutuhan Solusi
M1	LLM memberikan jawaban keliru atau halusinasi pada domain Framework & Library	G1: Absennya <i>grounding</i> eksternal	Jawaban di- <i>ground</i> pada konten SO terverifikasi
M2	Kualitas jawaban tidak mencerminkan konsensus komunitas SO	G2: Sinyal kepercayaan tidak dieksploitasi	Bobot kepercayaan (<i>upvote, accepted</i>) dikodekan dalam KG
M3	Tidak dapat menelusuri dari mana jawaban berasal	G1 + G2: Tidak ada mekanisme <i>citation</i>	Setiap jawaban menyertakan referensi ke SO thread
M4	Jawaban tidak dapat menangani pertanyaan relasional kompleks	G3: Tidak ada penalaran berbasis struktur graf	Traversal graf untuk menghubungkan entitas terkait
M5	Pengetahuan LLM tidak mencakup versi <i>library</i> terbaru	G1 + G2: <i>Knowledge cutoff</i> tanpa mekanisme pembaruan	Pembaruan KG tanpa <i>re-training</i> LLM

traversal dan *ranking* konteks yang diambil. Traversal multi-hop diperlukan untuk menjawab pertanyaan yang menghubungkan beberapa entitas teknis. Ini merupakan kebutuhan inti yang menjawab G2 dan G3.

- F5.** *Grounded Generation* dengan *Citation*. Berdasarkan konteks yang diambil melalui *graph-guided retrieval*, sistem harus membangkitkan jawaban menggunakan LLM yang secara eksplisit dibatasi oleh konteks tersebut (*context-faithful generation*), dilengkapi dengan kutipan ke SO thread sumber. Ini menjawab G1.
- F6.** Protokol Evaluasi Komparatif Tiga Kondisi. Sistem harus menyediakan *pipeline* evaluasi yang dapat membandingkan tiga kondisi secara *apple-to-apple*: (a) LLM murni sebagai *baseline* (DaSilva2025), (b) RAG konvensional sebagai *ablation*, dan (c) *GraphRAG Framework* yang diusulkan menggunakan dataset evaluasi yang sama.
- F7.** Pembaruan KG Inkremental. Sistem harus mendukung penambahan konten SO baru ke dalam KG tanpa *re-training* model LLM, sehingga pengetahuan dapat diperbarui secara efisien seiring perkembangan ekosistem *library*.

Tabel III.3 Kebutuhan nonfungsional *GraphRAG Framework*.

No.	Kategori	Deskripsi	Kriteria Sukses
NF1	Akurasi	Jawaban lebih faktual dari kondisi <i>baseline</i>	<i>Hallucination rate</i> lebih rendah dari DaSilva2025<empty citation> secara statistik signifikan ($\alpha = 0,05$)
NF2	Keterlacakan	Setiap jawaban memiliki referensi ke SO thread	100% jawaban dengan ≥ 1 kutipan sumber
NF3	Latensi	Waktu respons yang wajar untuk pengguna	≤ 15 detik per kueri pada konfigurasi pengujian standar
NF4	Reprodusibilitas	Hasil evaluasi dapat direproduksi	Dataset evaluasi dan kode dipublikasikan secara terbuka
NF5	Modularitas	Komponen KG, <i>retrieval</i> , dan LLM dapat dikonfigurasi	Setiap komponen dapat diganti melalui file konfigurasi
NF6	Skalabilitas	Mendukung subset SORD skala menengah	KG dengan > 10.000 simpul tanpa degradasi latensi yang signifikan

III.2.3 Kebutuhan Nonfungsional

III.3 Analisis Pemilihan Solusi

III.3.1 Alternatif Solusi

Berdasarkan identifikasi masalah dan kebutuhan di atas, tiga kondisi dipertimbangkan sekaligus membentuk *framework* evaluasi komparatif penelitian ini.

- F1.** Kondisi A — LLM Murni (*Baseline* **DaSilva2025<empty citation>**). LLM menjawab kueri pengguna langsung dari *parametric knowledge* tanpa augmentasi konteks eksternal. Ini adalah kondisi yang dievaluasi secara empiris oleh **DaSilva2025<empty citation>** dan **Kabir2024<empty citation>**, dan menjadi titik acuan pengukuran *improvement*. Keterbatasan yang sudah terdokumentasi adalah halusinasi pada domain Framework & Library, absennya *citation*, kegagalan pada pertanyaan relasional kompleks, ketidakmampuan mengadaptasi pengetahuan tanpa *re-training*.
- F2.** Kondisi B — RAG Konvensional (*Ablation Study*). Konten SO dari SORD diindeks sebagai *flat chunks* ke dalam *vector store*. Pada waktu inferensi, potongan teks paling mirip secara semantik diambil dan dimasukkan ke dalam *prompt* LLM. Keterbatasan arsitektural adalah tidak mengeksplorasi struktur relasional KG, sinyal kepercayaan komunitas SO (G2) tidak dimanfaatkan, ti-

dak mampu menjawab kueri multi-hop, rentan terhadap *knowledge noise* dari *chunk* yang kurang relevan (Zhou2026). Studi Nashid2023<empty citation> menunjukkan peningkatan yang menjanjikan tetapi terbatas karena ketiadaan struktur relasional.

F3. Kondisi C — *GraphRAG Framework* (Solusi yang Diusulkan). Konten SO dari SORD direpresentasikan sebagai KG berbobot di mana sinyal kepercayaan dikodekan secara eksplisit. Mekanisme *graph-guided retrieval* memanfaatkan struktur relasional dan bobot kepercayaan untuk mengambil konteks yang lebih relevan. Jawaban di-ground pada konteks tersebut dan dilengkapi *citation* ke SO thread. Keunggulan arsitektural yaitu menjawab G1 (grounding arsitektural berbasis SO), G2 (sinyal kepercayaan sebagai bobot KG), dan G3 (*GraphRAG* pada KG dari komunitas teknis bervalidasi sosial). Studi Zhu2026<empty citation>, Wang2026<empty citation>, dan Zhou2026<empty citation> mengkonfirmasi superioritas pendekatan berbasis KG berbobot dibandingkan RAG konvensional pada berbagai domain teknis.

III.3.2 Analisis Penentuan Solusi

Tabel ?? menyajikan matriks komparasi ketiga kondisi terhadap kebutuhan yang telah diidentifikasi. Matriks ini sekaligus mendefinisikan posisi *improvement* yang diklaim penelitian ini terhadap kondisi A dan B.

Berdasarkan matriks ini, **Kondisi C (*GraphRAG Framework*)** dipilih sebagai solusi yang dikembangkan. Gambar ?? menyajikan ringkasan visual dari perbandingan ketiga kondisi tersebut, sekaligus menunjukkan posisi penelitian ini terhadap baseline DaSilva2025<empty citation> dan kondisi pembanding RAG konvensional. Kondisi A dan B tetap diimplementasikan sebagai pembanding dalam protokol evaluasi komparatif (kebutuhan F6), memungkinkan pengukuran *improvement* yang terukur dan dapat direproduksi.

III.4 Desain Konseptual *GraphRAG Framework*

III.4.1 Gambaran Umum Arsitektur

Arsitektur *GraphRAG Framework* yang diusulkan diturunkan langsung dari kerangka konseptual empat jalur yang disusun pada Bab ?? (Gambar ??). Empat jalur tersebut diimplementasikan sebagai komponen sistem yang saling berinteraksi:

1. Jalur 1 — Konstruksi KG Berbobot dari Stack Overflow, merespons G2 dan G3 dengan membangun representasi terstruktur konten SO yang mengkodekan sinyal kepercayaan komunitas sebagai bobot eksplisit.
2. Jalur 2 — *Graph-Guided Retrieval*, merespons G3 dengan memanfaatkan

Tabel III.4 Matriks komparasi ketiga kondisi terhadap kebutuhan sistem. Kondisi C adalah solusi yang diusulkan.

Kebutuhan / Kriteria	Kondisi A LLM Murni	Kondisi B RAG Konvensional	Kondisi C GraphRAG
Reduksi halusinasi (M1)	□	Sebagian	Ya
Bobot sinyal kepercayaan SO (M2, F1)	□	□	Ya
Keterlacakan sumber / <i>citation</i> (M3, F5)	□	Sebagian	Ya (eksplisit)
Penalaran relasional / multi-hop (M4, F4)	□	□	Ya
Pembaruan tanpa <i>re-training</i> (M5, F7)	□	Ya	Ya
Kinerja pada domain Framework & Library	Lemah	Sedang	Target terbaik
Konsistensi dengan temuan G1 (Bab II)	Tidak	Tidak	Ya
Konsistensi dengan temuan G2 (Bab II)	Tidak	Tidak	Ya
Konsistensi dengan temuan G3 (Bab II)	Tidak	Tidak	Ya
Kesesuaian keseluruhan	Rendah	Sedang	Tinggi

struktur relasional KG untuk pengambilan konteks yang sensitif terhadap bobot kepercayaan.

3. Jalur 3 — *Grounded Generation*, merespons G1 dengan membatasi pengembangan jawaban LLM pada konteks terverifikasi dari KG SO, dilengkapi *citation*.
4. Jalur 4 — Evaluasi dan Validasi, memastikan *improvement* terhadap kondisi A dan B dapat diukur secara kuantitatif.

Gambar ?? mengilustrasikan hubungan antar komponen dalam arsitektur *framework*.

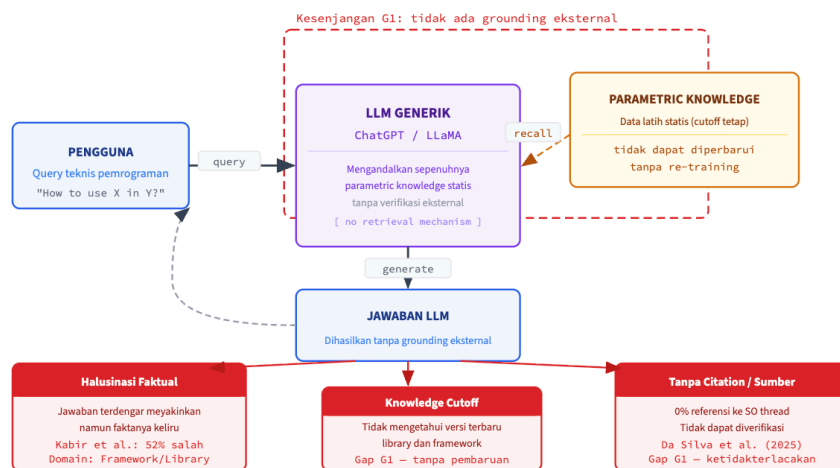
III.4.2 Desain *Knowledge Graph* Berbobot dari SORD

KG berbobot merupakan komponen inti yang menjawab G2 (memanfaatkan sinyal kepercayaan komunitas) dan G3 (membangun GraphRAG pada KG dari komunitas teknis bervalidasi sosial). Desain KG mencakup tipe simpul, tipe sisi, dan skema pembobotan yang dijabarkan berikut.

PERBANDINGAN APPLE-TO-APPLE — DATASET YANG SAMA		
Dataset SORD — Da Silva et al. (2025) + zenodo.15086541 Ground Truth: Jawaban Diterima Stack Overflow LLM yang sama di semua kondisi		
Kondisi A — LLM Murni Baseline Penelitian Da Silva et al. (2025) Kabir et al. (2024)	Kondisi B — RAG Konvensional Ablation Study Vector-only RAG Nashid et al. (2023)	Kondisi C — GraphRAG Framework Solusi yang diusulkan (Penelitian ini) Zhu (2026) Wang (2026) Zhou (2026) Menjawab Kesenjangan G1 + G2 + G3
Reduksi Halusinasi (M1) Kabir et al.: 52% jawaban keliru (LLM murni)	Tidak Parametric knowledge saja	Ya — grounded generation Jawaban dikontrol oleh KG SORD
Sinyal Kepercayaan SO (M2, G2) upvote, accepted answer, reputasi kontributor	Tidak dieksploitasi	Ya — bobot eksplisit pada KG w_accepted, w_score, w_reputation
Keterlacakan Sumber / Citation (M3, G1) Referensi ke SO thread — 0% pada kondisi A	0% referensi Da Silva et al. (2025)	Ya — eksplisit Kutipan ke SO thread pada setiap jawaban
Penalaran Relasional Multi-hop (M4, G3) Traversal KG untuk menghubungkan entitas terkait	Tidak	Ya — k-hop traversal Menghubungkan entitas lintas thread
Pembaruan Tanpa Re-training (M5) Tambah konten SO baru tanpa fine-tuning LLM	Tidak	Ya — KG inkremental Tanpa re-training LLM
Akurasi Domain Framework dan Library Domain paling lemah pada baseline (Da Silva et al.)	Lemah ~41% gagal pada domain ini	Target Terbaik Sinyal kepercayaan + traversal relasional
Kesesuaian Keseluruhan	RENDAH	TINGGI Solusi yang Dipilih

METRIK EVALUASI — DIUKUR PADA KETIGA KONDISI DENGAN LLM YANG SAMA						
Cosine Similarity Metrik utama baseline Da Silva et al. (2025)	ROUGE-1 / 2 / L Kemiripan n gram vs ground truth SO	BERTScore Kemiripan semantik berbasis embedding	Hallucination Rate LLM-as-Judge scoring Gu et al. (2026)	Attribution Coverage Persentase jawaban dengan kutipan SO	LLM-as-Judge Faithfulness, Comprehensiveness	Uji Statistik Paired t-test alpha = 0.05
Target Improvement: Kondisi C lebih baik dari Kondisi B dan A pada Hallucination Rate, Attribution Coverage, dan Akurasi Domain Framework dan Library						

Gambar III.2 Matriks komparasi tiga kondisi evaluasi terhadap sembilan kriteria kebutuhan sistem. Kondisi A (LLM Murni) merupakan *baseline* (DaSilva2025), Kondisi B (RAG Konvensional) merupakan *ablation study*, dan Kondisi C (*GraphRAG Framework*) merupakan solusi yang diusulkan. Ketiga kondisi akan dievaluasi menggunakan dataset yang sama dengan metrik yang kompatibel terhadap hasil yang dilaporkan oleh DaSilva2025^{<empty citation>}.



Gambar III.3 Arsitektur *GraphRAG Framework* berdasarkan kerangka konseptual empat jalur dari Bab ?? . Label G1–G3 menunjukkan kesenjangan yang dijawab oleh setiap komponen.

III.4.2.0.1 Tipe Simpul.

- (a) Question — merepresentasikan pertanyaan SO; atribut: *id*, *title*, *body*, *score*, *viewCount*, *creationDate*, *domainTag*.
- (b) Answer — merepresentasikan jawaban SO; atribut: *id*, *body*, *score*, *isAccepted*, *authorReputation*.
- (c) Tag — merepresentasikan teknologi atau domain; atribut: *name*, *questionCount*.
- (d) Concept — entitas teknis yang diekstrak dari teks (nama *library*, tipe *error*, metode API); atribut: *name*, *type*, *frequency*.

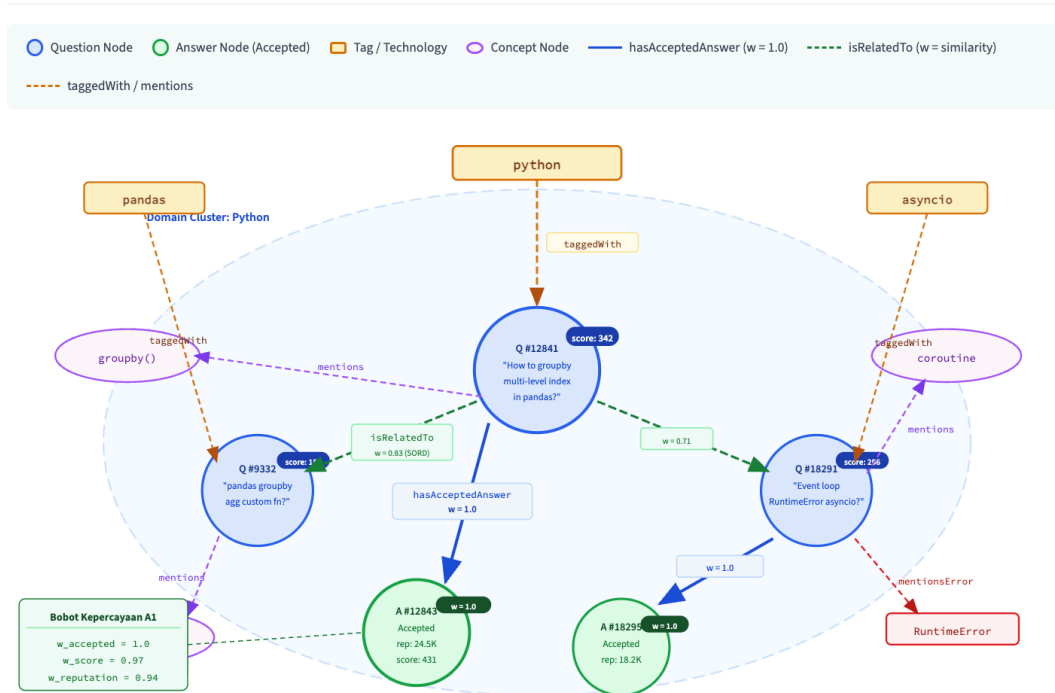
III.4.2.0.2 Tipe Sisi dan Skema Bobot Kepercayaan. Bobot pada sisi KG mengkodekan sinyal kepercayaan komunitas SO secara eksplisit. Tabel ?? merumuskan operasionalisasi sinyal-sinyal tersebut.

Tabel III.5 Operasionalisasi sinyal kepercayaan Stack Overflow sebagai bobot sisi pada *knowledge graph*.

Tipe Sisi	Arah	Makna Semantik	Bobot w
hasAcceptedAnswer	$Q \rightarrow A$	Jawaban yang diterima sebagai solusi terbaik oleh penanya	$w = 1,0$ (deterministic)
hasAnswer	$Q \rightarrow A$	Jawaban lain dengan skor tinggi	$w = \text{normalize}(\text{score})$
taggedWith	$Q \rightarrow \text{Tag}$	Pertanyaan dikategorikan dalam domain teknologi tertentu	$w = 1,0$
isRelatedTo	$Q \leftrightarrow Q$	Pasangan rekomendasi dalam dataset SORD	$w = \text{similarity}(q_i, q_j)$
mentions	$Q/A \rightarrow \text{Concept}$	Pertanyaan atau jawaban menyebut entitas teknis tertentu	$w = \text{TF-IDF}(\text{concept}, \text{doc})$
authorTrust	$A \rightarrow \text{Author}$	Kepercayaan terhadap kontributor jawaban	$w = \text{normalize}(\text{reputation})$

Untuk memberikan gambaran konkret tentang struktur KG yang dihasilkan, Gam-

bar ?? menyajikan contoh subgraf KG berbobot dari domain Python. Subgraf ini mengilustrasikan bagaimana tipe simpul (Question, Answer, Tag, Concept) dan tipe sisi (hasAcceptedAnswer, isRelatedTo, taggedWith, mentions) terhubung satu sama lain, beserta nilai bobot kepercayaan yang dikodekan secara eksplisit pada setiap sisi.



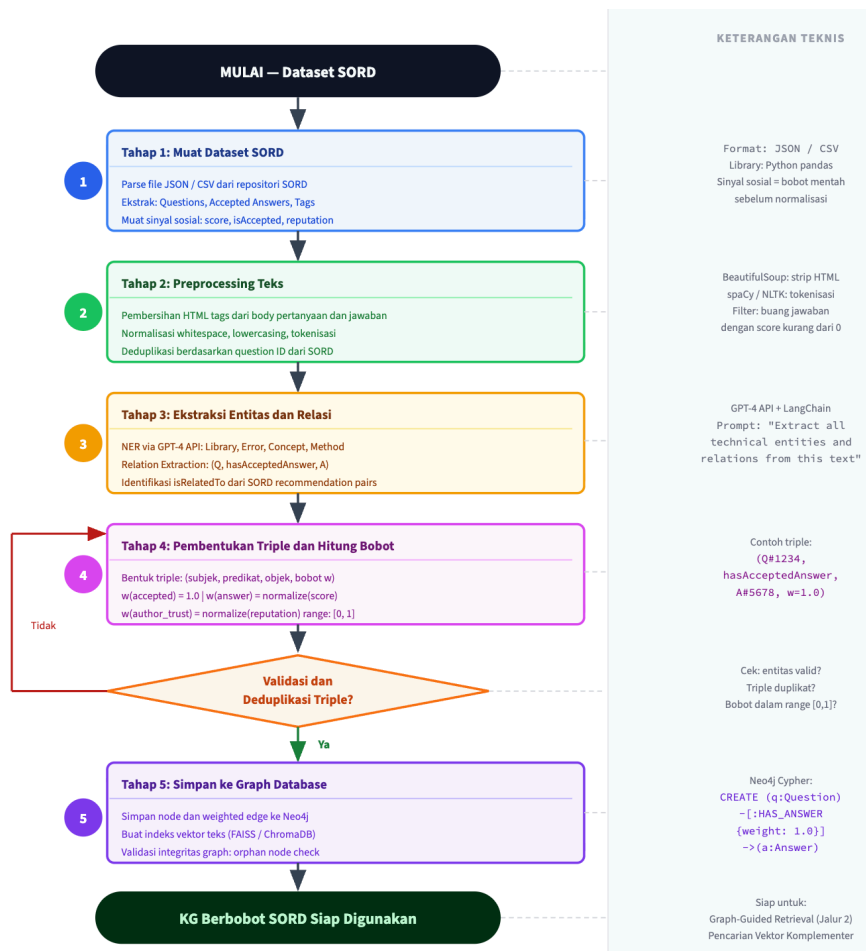
Gambar III.4 Contoh subgraf *knowledge graph* berbobot dari dataset SORD pada domain Python. Setiap sisi graf dilengkapi dengan bobot kepercayaan komunitas Stack Overflow (w) yang dikodekan secara eksplisit, meliputi bobot hasAcceptedAnswer ($w = 1.0$), hasAnswer ($w = \text{normalize}(\text{score})$), dan isRelatedTo ($w = \text{similarity}(q_i, q_j)$) dari pasangan rekomendasi SORD.

III.4.2.0.3 Proses Konstruksi KG. Gambar ?? mengilustrasikan alur konstruksi KG berbobot dari dataset SORD.

III.4.3 Desain Mekanisme *Graph-Guided Retrieval*

Mekanisme *graph-guided retrieval* merupakan komponen yang menjawab G3 dengan mengeksplorasi struktur relasional KG berbobot. Terdapat dua moda pengambilan konteks yang dipilih berdasarkan karakteristik kueri:

1. Moda 1 — Traversal Berbasis Entitas (untuk Kueri Spesifik). Jika kueri mengandung entitas teknis yang dapat dipetakan ke simpul KG (misalnya nama *library*, nama fungsi, atau tipe *error*), sistem melakukan: (i) NER pada teks kueri, (ii) *entity linking* ke simpul KG, (iii) traversal multi-hop (default $k =$



Gambar III.5 Flowchart konstruksi *knowledge graph* berbobot dari dataset SORD.

2) mengikuti sisi berbobot tertinggi, dan (iv) pengambilan jawaban diterima yang terhubung ke subgraf yang ditemukan. Bobot kepercayaan digunakan untuk *ranking* konteks yang diambil.

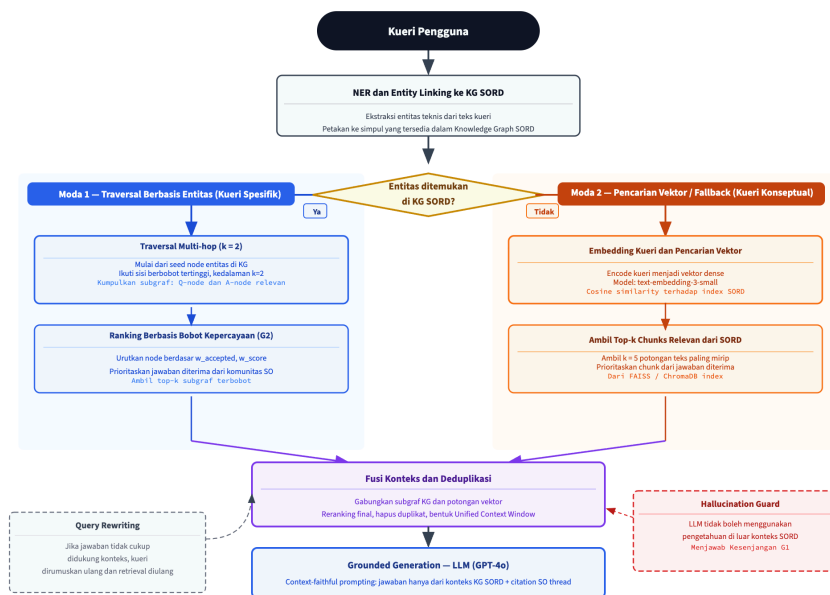
2. Moda 2 — Pencarian Vektor Komplementer (untuk Kueri Konseptual). Jika kueri tidak mengandung entitas yang dapat di-link ke KG, sistem melakukan pencarian semantik berbasis kemiripan kosinus pada *vector store* teks SORD. Hasil pencarian vektor juga digunakan sebagai *fallback* ketika traversal graf menghasilkan subgraf yang terlalu kecil.

Gambar ?? mengilustrasikan alur pemrosesan kueri secara lengkap.

III.4.4 Desain Evaluasi

Evaluasi dirancang untuk mengukur *improvement* yang diklaim dalam hipotesis penelitian (Bab ??) secara terukur dan dapat direproduksi. Desain protokol mengikuti prinsip *apple-to-apple comparison*:

1. Dataset Evaluasi. Subset pertanyaan dari dataset **DaSilva2025**<empty citation>



Gambar III.6 Flowchart alur pemrosesan kueri pada *GraphRAG Framework*. Moda 1 digunakan ketika entitas kueri dapat dipetakan ke KG; Moda 2 digunakan sebagai *fallback*.

(repositori chat-stack / zenodo.15086541) digunakan sebagai *test set*. Jawaban diterima Stack Overflow dijadikan *ground truth*. Ini memastikan komparabilitas langsung dengan angka-angka yang dilaporkan oleh DaSilva2025<empty citation> sebagai *baseline*.

2. Kontrol Variabel LLM. Ketiga kondisi menggunakan model LLM yang sama untuk mengeliminasi perbedaan model sebagai variabel perancu. Perbedaan satu-satunya antara kondisi A, B, dan C adalah mekanisme *retrieval* dan *grounding* yang digunakan.

3. Metrik Evaluasi. Tabel ?? merinci metrik yang digunakan beserta alasannya.

III.5 Ringkasan Desain

Tabel ?? merangkum komponen teknis *GraphRAG Framework* beserta kaitannya dengan kesenjangan yang dijawab dan kebutuhan yang dipenuhi.

Desain ini mengakomodasi seluruh kebutuhan fungsional F1–F7 dan kebutuhan non-fungsional NF1–NF6 yang telah diidentifikasi. Ia juga memastikan konsistensi antara argumen *research gap* pada Bab ??, hipotesis yang dirumuskan pada Bab ??, dan keputusan rancangan yang dibuat di bab ini. Perancangan rinci komponen, implementasi, dan hasil evaluasi komparatif dibahas pada bab-bab selanjutnya.

Tabel III.6 Metrik evaluasi komparatif *GraphRAG Framework*.

Metrik	Alasan Pemilihan	Kondisi A/B/C
<i>Cosine Similarity</i>	Metrik utama DaSilva2025 <empty citation>; memastikan komparabilitas <i>apple-to-apple</i>	Semua
ROUGE-1/2/L	Kemiripan berbasis <i>n-gram</i> dengan jawaban <i>ground truth</i>	Semua
BERTScore	Kemiripan semantik berbasis <i>embedding</i> kontekstual	Semua
<i>Hallucination Rate</i>	Persentase klaim yang tidak terdukung dalam jawaban (dinilai LLM-as-Judge (Gu2026))	Semua
<i>Attribution Coverage</i>	Persentase jawaban yang mencantumkan ≥ 1 kutipan SO thread	Semua (C sebagai target 100%)
<i>LLM-as-Judge Score</i>	Penilaian kualitas holistik: <i>faithfulness, comprehensiveness,</i> <i>relevance</i>	Semua
Uji Statistik	<i>Paired t-test</i> ($\alpha = 0,05$) untuk signifikansi perbedaan antar kondisi	A vs C, B vs C

Tabel III.7 Ringkasan komponen *GraphRAG Framework* dan kaitannya dengan kesenjangan penelitian

No.	Komponen	Fungsi	Gap	Teknologi Kandidat
1	Data Loader	Memuat dan memarsing dataset SORD	G2, G3	Python pandas / JSON
2	Preprocessor	Pembersihan, <i>deduplication</i> , dan tokenisasi	G2	spaCy / NLTK
3	KG Builder (NER)	Ekstraksi triple (<i>s, p, o</i>) dari teks	G3	GPT-4 API + LangChain
4	Weight Calculator	Komputasi bobot kepercayaan SO pada setiap sisi graf	G2	Python (normalisasi numerik)
5	Graph Database	Penyimpanan dan proses traversal <i>knowledge graph</i> berbobot	G2, G3	Neo4j / NetworkX
6	Embedding Model	Pengkodean teks untuk pencarian berbasis vektor	—	text-embedding-3-small
7	Vector Store	Indeks dan pencarian top- <i>k</i>	—	FAISS / ChromaDB
8	Retrieval Engine	<i>Graph-guided retrieval</i> dan pencarian berbasis vektor	G2, G3	Custom (Python)
9	LLM Generator	<i>Grounded generation</i> dan mekanisme <i>citation</i>	G1	GPT-4o (API OpenAI)
10	Evaluator	Komputasi metrik evaluasi komparatif	G1–G3	ROUGE, BERTScore, DeepEval

